

安徽信息工程学院学院

教 案

2024~2025 学 年 秋 季 学 期

开 课 学 院	计算机与软件工程学院
教研室(课程组)	人工智能教研室
课 程 代 码	CSE320
课 程 名 称	数据库原理与应用
授 课 对 象	人工智能 2301、2302 班
主 讲 教 师	冯璜
职 称	

二〇二四年八月

教 案（首页）

课 程 名 称	数据库原理与应用	课程编码	CSE320	总计：100 学时 课内：48 学时 理论：32 学时 实验：0 学时 上机：16 学时 课外：52 学时 其他：0 学时
		学 分	2.5	
课 程 类 别 (请打 √)	☐ 通识课 ☒ 学科基础课 ☒ 专业核心课 ☐ 专业方向课 ☐ 选修课 ☐ 集中实践教学环节			
任 课 教 师	冯璵	职 称		
授 课 对 象	专业班级： 人工智能 2301 班， 人工智能 2302 共 2 个教学班			
教 材	[1] 《数据库原理及应用（MySQL 版）》，李月军、付良廷主编，清华大学出版社，2019.			
主 要 参 考 资 料	[1] 《数据库系统概论》（第五版），王珊、萨师煊主编，高等教育出版社，2014. [2] 《MySQL 数据库应用与实践教程》，卜耀华、石玉芳主编，清华大学出版社，2017.			
教 学 重 点 难 点	教学重点： 1、数据模型与数据库体系结构 2、数据定义、增删改查、索引与视图的 SQL 语句实现 3、数据库完整性及 SQL 语句实现与关系代数运算 4、数据库的安全性及 MySQL 语句实现 5、并发控制及封锁 6、故障恢复的类别及恢复策略 7、使用 E-R 模型实现数据库的概念设计 8、关系模型的规范化 9、数据库设计 教学难点： 1、SQL 语言实现数据复杂查询 2、关系代数组合运算 3、并发控制 4、关系规范化：最小函数依赖集、函数依赖关系、求候选键、关系模式的规范化 5、数据库设计：逻辑结构向关系模型的转换			

《数据库原理与应用》 课程教案（第 1 次课第 1 章）

授课方式 (请打√)	理论课 ☒ 讨论课 ☒ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	4	课 外	0
教学单元（教学章、节或主题）：数据库系统的基本原理					
目的、要求（分了解、理解、掌握三个层次） 知识与能力： 掌握下述内容：数据、数据库、数据库管理系统、数据库系统的概念；数据管理技术的产生和发展；数据库系统的特点；数据模型（数据模型的组成要素、常用数据模型、概念模型、层次模型、网状模型、关系模型） 价值塑造： （1）提升工作责任心与担当精神；（2）增强规范与规则意识。					
重点 概念模型、数据模型					
难点 关系模型的概念					
教学步骤 签到 → 课程介绍 → 课程导入 → 精讲课堂内容 → 测验与小结					
教具及教学手段 举例讲解、多媒体讲解、博思智慧学习平台教学工具相结合					
作业和思考题： 课后作业： 教材第 1 章课后习题 思考题： 概念模型三个组成部分是什么，分别用什么图形表示？					
教 学 后 记					

教 学 过 程

板 书 或 旁 注

教学环节	课程介绍	新课讲解	课堂实践	每课小结	布置作业
时间分配	10	50	25	3	2

一、课程介绍：

数据库是现代信息社会的基石。几乎所有的计算机应用软件都构建于数据库系统之上。

数据库原理与应用是一门理论性较强的专业核心课程。本课程的主要任务是，掌握数据库系统的概念、基本原理 、基本方法以及有关的应用，同时介绍一种大型的数据库管理系统（MySQL）。重点掌握关系数据库的基本理论与基本操作，通过对关系数据库标准语言 SQL 的学习，了解掌握关系数据库的基本的使用，培养学生对有关实际问题的系统分析能力和设计能力。主要内容有:数据库系统、数据库管理系统、概念模型与 E-R 图；关系数据库、关系模型、关系代数方法、关系数据库标准语言 SQL、关系数据理论；数据库设计、实施与维护、数据库恢复技术、数据库安全性、数据库完整性和并发控制等。通过上机实践， 进一步加深理论知识，使学生能针对具体问题编制相应的数据库应用程序，今后能独立开发面向应用的数据库系统的应用程序，进一步提高学生的程序设计和调试能力。

二、成绩考核方法：

本课程为考试课程,其成绩组成为平时成绩(包括出勤 20%、线上考核 20%、阶段测试 20%、团队项目 40%)占 50%，期末考试占 50%。

三、内容

常用数据库：

Rank						DBMS	Database Model	Score		
Feb 2022	Jan 2022	Feb 2021			Feb 2022			Jan 2022	Feb 2021	
1.	1.	1.				Oracle 🏆	Relational, Multi-model 🔗	1256.83	-10.05	-59.84
2.	2.	2.				MySQL 🏆	Relational, Multi-model 🔗	1214.68	+8.63	-28.69
3.	3.	3.				Microsoft SQL Server 🏆	Relational, Multi-model 🔗	949.05	+4.24	-73.88
4.	4.	4.				PostgreSQL 🏆🌐	Relational, Multi-model 🔗	609.38	+2.83	+58.42
5.	5.	5.				MongoDB 🏆	Document, Multi-model 🔗	488.64	+0.07	+29.69
6.	6.	↑7.				Redis 🏆	Key-value, Multi-model 🔗	175.80	-2.18	+23.23
7.	7.	↓6.				IBM Db2	Relational, Multi-model 🔗	162.88	-1.32	+5.26
8.	8.					Elasticsearch	Search engine, Multi-model 🔗	162.29	+1.54	+11.29
9.	9.	↕11.				Microsoft Access	Relational	131.26	+2.31	+17.09
10.	10.	↓9.				SQLite 🏆	Relational	128.37	+0.94	+5.20
11.	11.	↕10.				Cassandra 🏆	Wide column	123.98	+0.43	+9.36
12.	12.	12.				MariaDB 🏆	Relational, Multi-model 🔗	107.11	+0.69	+13.22
13.	13.	13.				Splunk	Search engine	90.82	+0.37	+2.28

近几年一直处于此次序不变，关系数据模型 Relational 占主流，同时近几年每种数据库在此基础上拓展其他的数据模型。

特点：

Massive （数据量大）

– Database systems are handling terabytes of data, sometimes even terabytes of data every day.数据库系统每天要处理数 tb 的数据，有时甚至是数 tb 的数据。

▪ Persistent （持久）

– The data in the database outlives the programs that execute on that data.
数据库中的数据比执行该数据的程序寿命更长。

▪ Safe （安全）

– It won't be lost or overwritten when there are failures-hardware failures, software ,power outages, malicious users .
它不会丢失或覆盖时，有故障-硬件故障，软件，停电，恶意用户。

▪ Multi-user （多用户）

– Many different users or applications to access the data concurrently. There's mechanisms in database systems called concurrency control.
多个不同的用户或应用同时访问数据。数据库系统中有一种机制叫做并发

控制

Convenient（方便）

– Physical Data Independence, the way that data is actually stored and laid out on disk is independent of the way that programs think about the structure of the data.

– High level query languages, at a very high level what information you want from the database.

– Declarative, in the query, you describe what you want out of the database but you don't need to describe the algorithm to get the data out.

物理数据独立性，即数据在磁盘上实际存储和布局的方式与程序考虑数据结构的方式无关。

-高级查询语言，在非常高的级别上，你想从数据库中获得什么信息。

-声明式，在查询中，你描述你想从数据库中得到什么，但你不需要描述算法来获取数据。

▪ Efficient（高效）

– Database systems have to do really thousands of queries or updates per second.

数据库系统必须每秒执行数千次查询或更新。

▪ Reliable（可靠）

– 99.99999 % up time is the type of guarantee that database management systems are making for their applications, banking system or your telecommunications

99.99999%的 up 时间是数据库管理系统为其应用程序、银行系统或你的电信所做的类型保证

第 1 章 数据库系统的基本原理

【课程导入】

讨论：

淘宝网的主要功能和特点：

- 1、系统可以对哪些数据进行存储
- 2、存储信息的总量是多还是少
- 3、系统可以对数据进行什么操作
- 4、数据的操作效率如何

【讲授内容】

直播，平台不一样，速度有快有慢

区分信息和数据：

1) 信息

信息是客观存在的，是关于现实世界事物的存在方式或运动状态反映的综合

例如，安信工的学生

信息与材料和能源一个层次,是人类社会赖以生存和发展的三大资源之一

2) 数据

数据是用来记录信息的可识别的符号，是信息的具体表现形式

例如：计算机中的一张二维表，该数据安信工学生的基本信息
其表现形式包括数字、文字、图形、图像和声音等等

3) 数据与信息的关系

数据是信息的符号表示或载体

信息则是数据的内涵，是对数据的语义解释

1.1 数据库系统概述

1.1.1 数据库系统的应用

电信业、银行业、金融业、销售业、联机的零售商、大学、航空业、人力资源、制造业。

1.1.2 数据库系统概念 1. 数据 (Data) 数据是数据库存储的基本对象。是描述现实世界中各种具体事物或抽象概念的、可存储并具有明确意义的符号记录。 2. 数据库 (DataBase, 简称为 DB) 简单的说, 数据库就是相互关联的数据集合。严格的说, 数据库是长期存储在计算机内、有组织的、可共享的大量数据的集合。数据库中的数据按一定的数据模型组织、描述和存储, 具有较小的冗余度、较高的数据独立性和易扩展性。 3. 数据库管理系统 (DataBase Management System, DBMS) 它的主要功能包括以下几个方面: 数据定义功能、数据操纵功能、数据控制功能、数据组织、存储和管理。 4. 数据库系统 (DataBase System, DBS) 数据库系统是指在计算机系统中引入数据库后的系统, 一般由数据库 (DB)、数据库管理系统 (DBMS)、应用系统和数据库管理员 (DBA) 构成。 5. 数据库应用系统 (DataBase Application System, DBAS) 数据库应用系统主要是指实现业务逻辑的应用程序。 1.1.3 数据库系统的用户 1. 数据库管理员 (DBA) DBA 的具体职责包括: 1) 参与数据库的设计; 2) 定义数据的安全性要求和完整性约束条件; 3) 日常维护; 4) 数据库的改进和重组、重构。 2. 数据库用户 根据工作性质及人员的技能, 可将数据库用户分为四类, 分别是最终用户、专业用户、数据库设计人员、应用程序员。 1.1.4 数据管理技术的发展阶段 1. 人工管理阶段 特点: 1) 数据不保存; 2) 数据不具有独立性; 3) 数据不共享 2. 文件管理系统阶段 文件管理系统阶段存储组织信息的主要弊端: 1) 数据的冗余和不一致; 2) 数据独立性差; 3) 数据孤立 3. 数据库系统阶段 数据库系统的特点: 1) 数据结构化; 2) 数据的共享性高, 冗余度低, 易扩充; 3) 数据独立性高; 4) 数据由 DBMS 统一管理和控制。 1.2 数据模型 1.2.1 数据抽象的过程*** 美国国家标准化协会 ANSI 根据数据抽象的级别定义了 4 种模型, 即概念模型、逻辑模型、外部模型、内部模型。 第 1 步: 根据用户需求, 设计数据库的概念模型, 这是一个“综合”的过程。 第 2 步: 根据转换规则, 把概念模型转换成数据库的逻辑模型, 这是一个“转换”的过程。 第 3 步: 根据用户的业务特点, 设计不同的外部模型, 给应用程序使用。也就是说, 应用程序使用的是数据库外部模型中的各个视图。 第 4 步: 数据库实现时, 要根据逻辑模型设计其内部模型。 1. 概念模型 现在采用的概念模型主要是实体-联系模型, 即 E-R 模型。 两个实体之间的联系可以分为三种: 1) 一对一联系 (1: 1); 2) 一对多联系 (1: n); 3) 多对多联系 (m: n) 2. 逻辑模型 逻辑模型有层次模型、网状模型和关系模型 3 种。	在黑板上通过画图讲解抽象过程
--	-----------------------

3. 外部模型

外部模型具有如下特点。

- 1) 外部模型是逻辑模型的一个逻辑子集。
- 2) 硬件独立，软件依赖。
- 3) 外部模型反映了用户使用数据库的观点。

4. 内部模型

内部模型又称为**物理模型**，是数据库最底层的抽象，它描述数据在磁盘上存储方式、存取设备和存取方法。

1.2.2 关系模型※※※

数据模型的三要素

- 1) 数据结构：是对系统静态特性的描述。
- 2) 数据操作：是对系统动态特性的描述。
- 3) 数据的完整性约束条件：在关系模型中，任何关系都必须满足实体完整性和参照完整性。

2. 关系数据模型的数据结构

1) 关系 (Relation)

一个关系就是一张规范的二维表。

2) 元组 (Tuple)

表中的一行即为一个元组。

3) 属性 (Attribute)

表中的一列即为一个属性，每个属性都有一个属性名。

4) 码或键 (Key)

也称为关键码或关键字。表中的某个属性或者属性的组合，能唯一的确定一个元组。

5) 关系模式

对关系的描述，一般表示为：

关系名 (属性 1, 属性 2, 属性 3,, 属性 n)。

例如：学生基本信息表 (学号, 姓名, 年龄, 性别, 籍贯)

关系的完整性约束条件包括三大类：

实体完整性；

参照完整性；

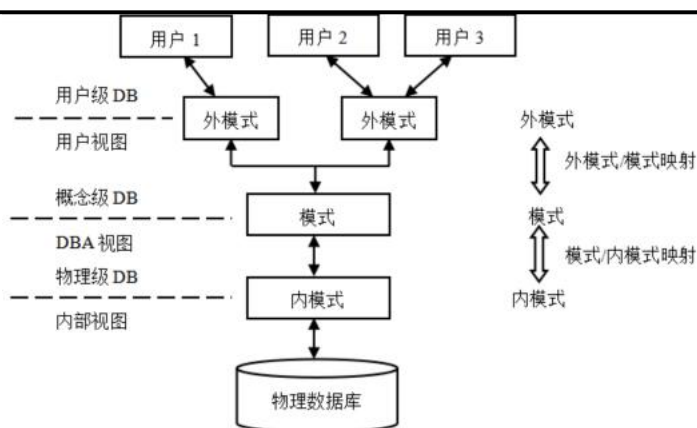
用户定义的完整性。

1.3 数据库体系结构

1. 数据库的三级体系结构※※※

数据库系统包括三级模式，即概念模式、外模式和内模式。

数据库系统的二级映射是：外模式/模式映射和模式/内模式映射。



2. 三级模式

1) 模式

模式又称为概念模式或逻辑模式，是数据库中全体数据的逻辑结构和特征的描述，是所有用户的公共数据视图。一个数据库只能有一个模式。

定义模式时不仅要定义数据的逻辑结构（如数据记录由哪些数据项构成，数据项的名字、类型、取值范围等），而且还要定义数据之间的联系，定义与数据有关的安全性、完整性要求。

2) 外模式

外模式又称为子模式或用户模式，是数据库用户（包括程序员和最终用户）能够看到和使用的局部数据的逻辑结构和特征的描述，是数据库用户的数据视图，是与某一应用有关的数据的逻辑表示。一个数据库可以有多个外模式。

外模式主要描述组成用户视图的各个记录的组成、相互关系、数据项的特征、数据的安全性和完整性约束条件。

3) 内模式

内模式又称为存储模式或物理模式，是数据物理结构和存储方式的描述，是数据在数据库内部的表示方式。一个数据库只能有一个内模式。

3. 数据库系统的二级映射※※※

数据库系统的二级映射是：外模式/模式映射和模式/内模式映射。

外模式/模式映射：

定义外模式与逻辑模式之间的对应关系；

每一个外模式都对应一个外模式 / 模式映射；

映射定义通常包含在各自外模式的描述中

模式/内模式映射：

逻辑模式 / 内模式映射定义了数据全局逻辑结构与

存储结构之间的对应关系；

数据库中模式 / 内模式映射是唯一的；

该映射定义通常包含在逻辑模式描述中；

数据库系统 3 个抽象级间通过两级映射进行相互转换，使得数据库抽象的 3 级模式形成一个统一的整体。

4. 数据独立性※※※

1) 物理独立性

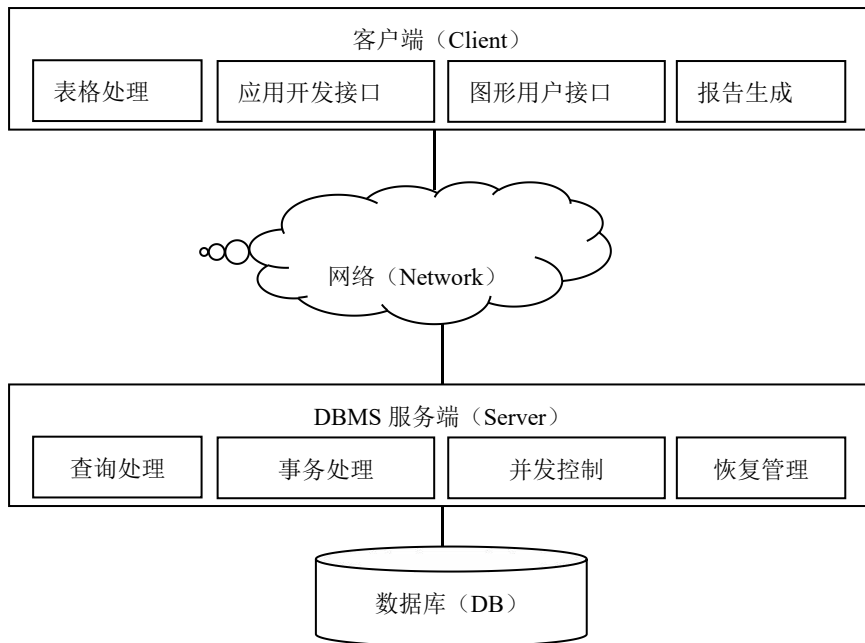
物理独立性是指用户的应用程序与存储在磁盘上的数据库中的数据是独立的。物理独立性是通过模式/内模式映射来实现的。

2) 逻辑独立性

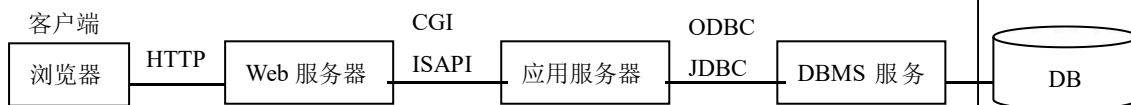
逻辑独立性是指用户的应用程序与逻辑结构是相互独立的。逻辑独立性是通过外模式/模式映射来实现的。

5. 数据库应用系统的开发架构

1) C/S 模式



2) B/S 模式



1.4 高级数据库系统**

分布式数据库系统

面向对象数据库系统

并行数据库系统

多媒体数据库系统

1.5 数据仓库技术与数据挖掘技术**

1.5.1 数据仓库

数据仓库 (Data Warehouse, DW) 是一个面向主题的、集成的、稳定的、随时间不断变化的数据集合，它用于支持经营管理中的决策制定过程。

1.5.2 联机分析处理

OLAP 可以对大量的多给数据进行动态合并和分析，是决策支持领域的一部分。

1.5.3 数据挖掘

数据挖掘 (Data Mining) 是从大量数据中提取隐含在其中的、人们事先不知道的但又可能有用的信息和知识的过程。

【课程小结】

本次课主要掌握 3 个内容，一是数据库的基本概念，二是数据库发展各阶段的特点，三是数据模型，重点掌握关系模型。

《数据库原理与应用》 课程教案（第 2 次课 4.1 4.2）

授课方式 (请打√)	理论课 ☒ 讨论课 ☒ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	0
教学单元（教学章、节或主题）： 第4章 关系模型基本理论					
目的、要求（分了解、理解、掌握三个层次） 1、了解关系数据库中的关系模型 2、熟悉关系数据结构及形式化定义的基本概念（关系、关系模式、关系数据库） 3、掌握关系的完整性。					
重点 1、关系的概念 2、关系模式的概念 3、关系的完整性					
难点 关系完整性实现					
教学步骤 签到 → 提问引起思考引入课程→ 理论讲解 → 小结与作业					
教具及教学手段 举例讲解、多媒体讲解、博思智慧学习平台教学工具相结合					
作业和思考题： 作业：教材第 4 章课后习题 思考题： 关系完整性包括哪三种？ 分别使用哪些命令短语实现？					
教 学 后 记					

定义 4.1 设 F 是关系 R 的一个或一组属性（但 F 不是 R 的主键），K 是关系 S 的主键。如果 F 与 K 相对应，则称 F 是关系 R 的外键，并称关系 R 为参照关系，关系 S 为被参照关系。而关系 R 和关系 S 可以是同一个关系。

规则 4.2 外键或者取空值（要求外键的每个属性均为空值），或者等于被参照关系中的主键的某个值。

参照完整性规则就是定义外键与主键之间的引用规则。

3. 用户定义的完整性

根据应用环境的特殊要求，关系数据库应用系统中的关系往往还应该满足一些特殊的约束条件。

规则 4.3 属性的取值应当满足用户定义的约束条件。

4.2.2 MySQL 提供的约束

1. 主键（PRIMARY KEY）约束

主键约束主要是针对主键，以保证主键值的完整性。主键约束要求主键值必须满足两个条件：

- （1）值唯一；
- （2）不能为空值。

2. 唯一约束

唯一约束主要是针对于候选键，以保证候选键值的完整性。唯一约束要求候选键满足 2 个条件：

- （1）值唯一；
- （2）可有一个且仅有一个空值。

3. 检查约束

检查约束是通过检查输入到表中的数据来维护用户定义的完整性的，即检查输入的每一个数据，只有符合条件的数据才允许输入到表中。

4. 外键约束

外键约束涉及到两个表，即主表和从表，从表是指外键所在的表，主表是指外键在另一张表中作为主键的表。

4.2.3 触发器

触发器创建的语法如下：

```
CREATE TRIGGER 触发器名
    BEFORE | AFTER
    INSERT | DELETE | UPDATE
    ON 表名
    FOR EACH ROW
    < 触发体 >
```

【课程小结】

本次课主要掌握数据的三种完整性及其实现。

平台操作演示

《数据库原理与应用》 课程教案（第 3 次课 4.3 4.4）

授课方式 (请打√)	理论课 ☒ 讨论课 ☒ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	0
教学单元（教学章、节或主题）：第4章 关系模型基本理论					
目的、要求（分了解、理解、掌握三个层次） 1、掌握关系代数传统的集合运算； 2、掌握关系代数专门的关系运算。					
重点 关系代数的运算					
难点 组合的关系代数运算					
教学步骤 签到 → 提问引起思考引入课程→ 理论讲解 → 小结与作业					
教具及教学手段 举例讲解、多媒体讲解、博思智慧学习平台教学工具相结合					
作业和思考题： 作业：教材第 4 章课后习题 思考题： 关系代数五种基本操作的运行符号分别是什么？					
教 学 后 记					

除操作适合于包含“对于所有的或全部的”语句的查询操作。

4.4 关系运算

4.4.1 元组关系运算

1. 原子公式三种形式

- (1) $R(t)$
- (2) $t[i] \theta s[j]$
- (3) $t[i] \theta c$ 或 $c \theta t[i]$

2. 公式的递归定义

- (1) 每个原子公式是公式。其中的元组变量是自由元组变量。
- (2) 如果 P_1 和 P_2 是公式，那么下面 3 个也为公式。
 - $\neg P_1$ ，如果 P_1 为真，则 $\neg P_1$ 为假。
 - $P_1 \vee P_2$ ，如果 P_1 和 P_2 中有一个为真或者同时为真，则 $P_1 \vee P_2$ 为真，仅当 P_1 和 P_2 同时为假时， $P_1 \vee P_2$ 为假。
 - $P_1 \wedge P_2$ ，如果 P_1 和 P_2 同时为真，则 $P_1 \wedge P_2$ 才为真，否则为假。
- (3) 如果 P 是公式，那么 $(\exists t)(P)$ 和 $(\forall t)(P)$ 也是公式。其中 t 是公式 P 中的自由元组变量，在 $(\exists t)(P)$ 和 $(\forall t)(P)$ 中称为约束元组变量。
 $(\exists t)(P)$ 表示存在一个元组 t 使得公式 P 为真； $(\forall t)(P)$ 表示对于所有元组 t 都使得公式 P 为真。

(4) 公式中各种运算符的优先级从高到低依次为： θ 、 $\exists$ 和 $\forall$ 、 $\neg$ 、 $\wedge$ 和 $\vee$ 。在公式外还可以加括号，以改变上述优先顺序。

(5) 公式只能由上述 4 种形式构成，除此之外构成的都不是公式。

3. 关系代数中 5 种基本运算用元组关系演算表达式的表达***

(1) 并

$$R \cup S = \{t \mid R(t) \vee S(t)\}$$

(2) 交

$$R \cap S = \{t \mid R(t) \wedge S(t)\}$$

(3) 投影

$$\prod_{i_1, i_2, \dots, i_k}(R) = \{t \mid (\exists u)(R(u) \wedge t[1]=u[i_1] \wedge t[2]=u[i_2] \wedge \dots \wedge t[k]=u[i_k])\}$$

例如，投影操作是 $\prod_{2,3}(R)$ ，那么元组表达式可写成：

$$\{t \mid (\exists u)(R(u) \wedge t[1]=u[2] \wedge t[2]=u[3])\}$$

(4) 笛卡尔积

$$R \times S = \{t \mid (\exists u)(\exists v)(R(u) \wedge S(v) \wedge t[1]=u[1] \wedge t[2]=u[2] \wedge t[3]=u[3] \wedge t[4]=v[1] \wedge t[5]=v[2] \wedge t[6]=v[3])\}$$

(5) 选择

$$\sigma_F(R) = \{t \mid R(t) \wedge F'\}$$

F' 是 F 的等价表示形式。

例如， $\sigma_{2='d'}(R)$ 可写成 $\{t \mid R(t) \wedge t[2]='d'\}$ 。

因为差运算也常用，所以下面给出差运算的元组关系演算表达式。

(6) 差

$$R - S = \{t \mid R(t) \wedge \neg S(t)\}$$

【课程小结】

本次课主要掌握通过关系代数的四个组合操作写出关系代数表达式，掌握关系代数，会更有利于对 SQL 语句的理解记忆。

黑板讲解

作业和思考题：

课后作业：

教材第 2 章课后选择题

思考题：

常用的数据类型包括哪几种？各数据类型的关键字是什么？

教 学

后 记

教 学 过 程						板 书 或 旁 注
教学环节	复习提问	新课讲解	课堂实践	每课小结	布置作业	
时间分配	5	68	10	5	2	
第二章 关系数据库标准语言 SQL 【课程导入】 问题：QQ 拥有的用户很多，如何将数亿条用户信息存储到计算机中哪？ 【讲授内容】 2.1 SQL 语言介绍 1. SQL 的特点 （1）综合统一；（2）高度非过程化；（3）面向集合的操作方式；（4）同一种语法结构提供两种使用方式；（5）语言简洁、易学易用 2. SQL 语言的组成 1) 数据定义语言（DDL） DDL 用来定义、修改、删除数据库中的各种对象。 2) 数据操纵语言（DML） DML 的命令用来查询、插入、修改、删除数据库中数据。包含用于查询数据（Select）、添加新行数据（Insert）、修改现有行数据（Update）、删除现有行数据（Delete）的语句等。 3) 数据控制语言（DCL） DCL 用于事务控制、并发控制、完整性和安全性控制等。 2.2 数据定义 1. 数据库的定义和删除*** 1) 创建数据库 CREATE DATABASE 数据库名; 2) 选择数据库 USE 数据库名; 3.) 删除数据库 DROP DATABASE 数据库名; 2. 数据类型*** 1) 字符串类型 常用的字符类型是 CHAR、VARCHAR 类型。 2) 数值类型 常用的数值类型是 INT、DECIMAL 类型。 3) 日期和时间类型 常用的日期和时间类型是 DATE、TIME、DATETIME 类型。 4) 布尔类型 布尔类型为 BOOLEAN 类型，只有两个值：TRUE 和 FALSE，即真值和假值。 3. 基本表的定义、删除和修改*** 1) 创建表 CREATE TABLE 表名						平台操作演示

(<列名> <数据类型> [DEFAULT <默认值>] [, ...]); 2) 利用子查询来创建表 CREATE TABLE <表名> SELECT 语句; 3) 修改表的结构 (1) 在一个表中增加一个新列 ALTER TABLE <表名> ADD [COLUMN] <列名> <数据类型> [DEFAULT <默认值>] (2) 修改一个表中已有的列 ALTER TABLE <表名> MODIFY [COLUMN] <列名> <数据类型> [DEFAULT <默认值>] (3) 从一个表中删除一列 ALTER TABLE <表名> DROP [COLUMN] <列名> 4. 截断表和删除表 1) 截断表 TRUNCATE TABLE <表名> 注意：使用上面的语句只删除了表中的所有数据行，但表的结构仍然保留。 2) 删除表 当不仅要删除表中的数据而且还要删除表的结构，可以使用 DROP TABLE 语句。该语句的格式为： DROP TABLE <表名> 2.3 数据查询 ※※※ SELECT 语句基本的语法如下： SELECT * <列名 列表表达式>[, <列名 列表表达式>]..... FROM <表名或视图名>[, <表名或视图名>]..... [WHERE <行条件表达式>] [GROUP BY <分组列名 1>[, <分组列名 2>]..... [HAVING <组条件 表达式>]] [ORDER BY <排序列名 1 [ASC DESC]> [, <排序列名 2 [ASC DESC]>].....] 两者不同： 1.SELECT 是等值连接在 condition 中要有等值连接条件 2.SELECT 结果不是集合是包，有重复元组 2.3.1 基本查询 查询仅涉及一个表，是一种最简单的查询操作： 一、 选择表中的若干列——SELECT 子句 二、 选择表中的若干元组——WHERE 子句 三、 对查询结果排序——ORDER BY 子句 四、 聚集函数 ——5 个集函数 五、 对查询结果分组——GROUP BY 子句	对应关系代数： π $A_1, A_2, \dots, A_n (\sigma$ condition ($R_1 \bowtie$ $R_2 \dots \bowtie R_m))$
---	---

六、对分组之后结果进行筛选——HAVING 子句 一. SELECT 子句的规定 SELECT 子句用于描述输出值的列名或表达式，其形式如下： SELECT [ALL DISTINCT] * <列名或列名表达式序列> 属投影运算 - 不消除重复行 ◆SELECT 子句查询指定属性列 - 查询指定列 - 查询全部列 - 查询经过计算的值 1.查询指定列 在 SELECT 子句中指定要查询的属性列 - 各个列的先后顺序可以与表中的逻辑顺序不一致 - 用户可以根据应用的需要改变列的显示顺序 2.查询全部列 ◆在 SELECT 关键字后面列出所有属性列名 ◆当列的显示顺序与其在基表中的顺序相同时，也可以简单地将属性列表指定为 *（* 代表全部列） 3.查询经过计算的值 SELECT 子句的<目标列表达式>可以为： 算术表达式 字符串常量 函数 列别名 二. 使用 WHERE 子句指定查询条件 1.消除取值重复的行 指定 DISTINCT 关键词，去掉表中重复的行 如果没有指定 DISTINCT 关键词，则缺省为 ALL DISTINCT 短语的作用范围是所有目标列 2.查询满足条件的元组——通过 WHERE Condition 子句实现		
运算符名称	符号及格式	说 明
算术比较判断	<表达式 1> θ <表达式 2> θ代表的符号有：<、<=、>、>=、<>或!=、=	比较两个表达式的值
逻辑比较判断	<比较表达式 1> θ <比较表达式 2> θ代表的符号按其优先级由高到低的顺序为： NOT、AND、OR	两个比较表达式进行非、与、或的运算
之间判断	<表达式> [NOT] BETWEEN <值 1> AND <值 2>	搜索（不）在给定范围内的数据
字符串模糊判断	<字符串> [NOT] LIKE <匹配模式>	查找（不）包含给定模式的值
		通配符（%代表任意长度

空值判断	<表达式> IS [NOT] NULL	判断某值是否为空值	的字符串; _代表任意单个字符) ESCAPE '<换码字符>' 短语对通配符进行转义
之内判断	<表达式> IN (<集合>)	判断表达式的值是否在集合内	

三、对查询结果排序——ORDER BY 子句

使用 ORDER BY 子句，可以按一个或多个属性列排序

升序：ASC；降序：DESC；缺省值为升序

当排序列含空值时

ASC：排序列为空值的元组最后显示

DESC：排序列为空值的元组最先显示

四、聚合函数

函 数	说 明
COUNT(*)	计算记录的个数
COUNT(<列名>)	对一列中的值计算个数
SUM(<列名>)	求某一列值的总和
AVG(<列名>)	求某一列值的平均值
MAX(<列名>)	求一列值的最大值
MIN(<列名>)	求一列值的最小值

使用聚合函数时，需要注意以下几点：

- 聚合函数只能出现在所查询的列、ORDER BY 子句、HAVING 子句中，而不能出现在 WHERE 子句、GROUP BY 子句中。
- 除了 COUNT(*) 之外，其他聚合函数（包括 COUNT(<列名>））都忽略对列值为 NULL 值的统计。

在列名前加 DISTINCT 则统计时去掉重复行

五、对查询结果分组 ——GROUP BY 子句

用途 - 细化集函数的作用对象

- 未对查询结果分组，集函数将作用于整个查询结果
- 对查询结果分组后，集函数将分别作用于每个组

使用 GROUP BY 子句分组：

- 分组方法：按指定的一列或多列值分组，值相等的为一组
- 使用 GROUP BY 子句后，SELECT 子句的列名列表中只能出现分组属性和集函数
- 作用对象是查询的中间结果表

六、对分组之后结果进行筛选 ——HAVING 子句

使用 HAVING 短语筛选最终输出结果

- 只有满足 HAVING 短语指定条件的组才输出

HAVING 短语与 WHERE 子句的区别：作用对象不同

- WHERE 子句作用于基表或视图，从中选择满足条件的元组
- HAVING 短语作用于组，从中选择满足条件的组

【课程小结】

本次课主要掌握 MySQL 下的常用数据类型，并能通过 SQL 语句实现数据库的创建及删除、表的创建、修改及删除。

《数据库原理与应用》 课程教案（第 5 次课 2.3~2.5）

授课方式 (请打√)	理论课 ☒ 讨论课 ☒ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	0
教学单元（教学章、节或主题）：第2章 关系数据库标准语言 SQL					
目的、要求（分了解、理解、掌握三个层次） 知识与能力： 1、掌握数据查询（连接查询；嵌套查询；集合查询） 2、掌握 SQL 的数据更新（插入数据；修改数据；删除数据） 价值塑造： （1）提升工作责任心与担当精神；（2）增强规范与规则意识。					
重点 1、数据查询（连接查询；嵌套查询；集合查询） 2、数据更新（插入数据；修改数据）					
难点 数据查询（嵌套查询；集合查询）；视图的使用。					
教学步骤 签到 → 课程介绍 → 理论讲解 → 小结与作业					
教具及教学手段 举例讲解、多媒体讲解、博思智慧学习平台教学工具相结合					
作业和思考题： 作业： 教材第 2 章课后习题 思考题： 什么情况下不能使用视图完成对表的插入、删除和更新操作？					
教 学 后 记					

教 学 过 程						板 书 或 旁 注
教学环节	复习提问	新课讲解	课堂实践	每课小结	布置作业	平台操作演示
时间分配	5	68	10	5	2	
第 2 章 关系数据库标准语言 SQL						
【课程导入】						
问题：用户登录界面注册新的用户，怎么将数据添加到数据库中？怎么样实现用户密码的修改？						
2.3 数据查询 ※※※						
2.3.2 连接查询						
同时涉及多个表的查询称为连接查询						
SQL 中连接查询的主要类型						
(1) 交叉连接（广义笛卡尔积）						
(2) 相等连接						
(3) 自身连接						
(4) 复合条件连接						
(5) 内连接						
(6) 外连接						
连接查询时的一些注意事项：						
① 要连接的表都要放在 FROM 子句中，表名之间用逗号分开，比如 FROM de						
tp,emp。						
② 为了书写方便，可以为表起别名，表的别名在 FROM 子句中定义，别名放在表名之后，它们之间用空格隔开。注意，别名一经定义，在整个查询语句中就只能使用表的别名而不能再用表名。						
③ 连 接 的 条 件 放 在 WHERE 子 句 中 ， 比 如 WHERE emp.deptno=dept.deptno。						
④ 如果多个表中有相同列名的列时，在使用这些列时，必须在这些列的前面冠以表名来区别，表名和列名之间用句号隔开。						
(1) 交叉连接（广义笛卡尔积）						
• 不带连接谓词的连接						
• 很少使用						
(2) 等值连接						
• 连接运算符为 = 的连接操作						
[<表名 1>.<列名 1>=<表名 2>.<列名 2>						
• 表名前缀						
任何子句中引用表 1 和表 2 中同名属性时，都必须加表名前						
缀						
引用唯一属性名时可以加也可以省略表名前缀						
(3) 自身连接						
一个表与其自己进行连接，称为表的自身连接						
表示方法：						

- 需要给表起别名以示区别 - 由于所有属性名都是同名属性，因此必须使用别名前缀 (4) 复合条件连接（多表连接） WHERE 子句中含多个连接条件时，称为复合条件连接 (5) 内连接 (6) 外连接 外连接与普通连接（内连接）的区别 • 普通连接操作只输出满足连接条件的元组 • 外连接操作以指定表为连接主体，将主体表中不满足连接条件的元组一并输出 • 左连接（LEFT JOIN）或左外连接（LEFT OUTER JOIN） 返回左表中的所有行，如果左表中行在右表中没有匹配行，则结果中右表中的列返回空值 • 右连接（RIGHT JOIN）或右外连接（RIGHT OUTER JOIN） 返回右表中的所有行，如果右表中行在左表中没有匹配行，则结果中左表中的列返回空值 • 全连接（FULL JOIN）或全外连接（FULL OUTER JOIN） 返回左表和右表中的所有行。当某行在另一表中没有匹配行，则另一表中的列返回空值 2.3.3 嵌套查询 查询块：一个 SELECT-FROM-WHERE 语句称为一个查询块 嵌套查询：将一个查询块嵌套在另一个查询块的 WHERE 子句或 HAVING 短语的条件中的查询称为嵌套查询 - WHERE condition - HAVING condition 1. 返回单值的子查询 2. 返回多值的子查询 1) 使用 IN 操作符的多值子查询 比较运算符 IN 的含义为子查询返回列表中的任何一个。IN 操作符比较子查询返回列表中的每一个值，并且显示任何相等的数据行。 2) 使用 ALL 操作符的多值子查询 ALL 操作符比较子查询返回列表中的每一个值。<ALL：为小于最小的；>ALL：为大于最大的。 3) 使用 ANY 操作符的多值子查询 ANY 操作符比较子查询返回列表中每一个值。<ANY：为小于最大 其中 ALL、ANY 必须与比较运算符结合使用

的；>ANY：为大于最小的。

4) 使用 EXISTS 操作符的多行查询

EXISTS 操作符比较子查询返回列表的每一行。使用 EXISTS 时应注意：外层查询的 WHERE 子句格式为：WHERE EXISTS；在内层子查询中必须有 WHERE 子句，给出外层查询和内层子查询所使用表的连接条件。

2.3.4 集合查询

当两个 SELECT 查询结果的结构完全一致时，可以对这两个查询执行并、交、差的运算，运算符为 UNION、INTERSECT 和 MINUS。

集合运算的格式为：

```
SELECT 语句 1
      UNION|INTERSECT|MINUS
SELECT 语句 2
```

【课程小结】

本次课主要掌握连接查询、嵌套查询和合并查询。

《数据库原理与应用》 课程教案（第 6 次课 第三章）

授课方式 (请打√)	理论课 ☒ 讨论课 ☒ 实践课 ☐ 习题课 ☒ 其他 ☐	学时安排			
		课 内	2	课 外	0
教学单元（教学章、节或主题）：第3章 数据库编程					
目的、要求（分了解、理解、掌握三个层次） 知识与能力： 1、掌握 SQL 的数据更新（插入数据；修改数据；删除数据） 2、掌握索引创建与删除 3、掌握视图（定义、查询、更新视图） 价值塑造： （1）提升工作责任心与担当精神； （2）增强规范与规则意识。					
重点 1、数据更新（插入数据；修改数据） 2、索引及视图					
难点 数据查询（嵌套查询；集合查询）；视图的使用；数据库安全性控制。					
教学步骤 签到 → 课程介绍 → 理论讲解 → 小结与作业					
教具及教学手段 举例讲解、多媒体讲解、博思智慧学习平台教学工具相结合					
作业和思考题： 作业： 教材第 2 章课后习题 思考题： 什么情况下不能使用视图完成对表的插入、删除和更新操作？					

教 学 后 记	
------------	--

教 学 过 程						板 书 或 旁 注
教学环节	复习提问	新课讲解	课堂实践	每课小结	布置作业	
时间分配	5	68	10	5	2	
第 2 章 关系数据库标准语言 SQL						
【课前回顾】						
数据简单查询						
数据复杂查询						
2.4 数据的维护 ※※※						
数据维护是指用 INSERT、DELETE、UPDATE 语句来插入、删除、更新数据库表中记录行的数据，由数据操作语言 DML 实现。						
2.4.1 插入数据						
1. INSERT 语句						
当往一个表中添加一行新的数据时，需要使用 DML 语言中的 INSERT 语句。该语句的基本语法格式如下：						
INSERT INTO 表名 [(列名 1[, 列名 2.....])] VALUES (值 1[, 值 2.....])						
2. 利用子查询向表中插入数据						
INSERT INTO 表名 [(列名 1[,列名 2.....])] SELECT 语句						
2.4.2 更新数据						
1. UPDATE 语句						
基本语法格式如下：						
UPDATE 表名 SET 列名=值[,列名=值,.....] [WHERE <条件>]						
【注意】						
MySQL 运行在 SAFE_UPDATES 模式下，该模式会导致非主键条件下无法执行 UPDATE 或 DELETE 命令，需要执行命令： SET SQL_SAFE_UPDATES=0; 修改数据库模式。						
2. 利用子查询修改记录						
2.4.3 删除数据						
1. DELETE 语句						
基本语法格式如下：						
DELETE [FROM] 表名 [WHERE <条件>]						
2. 利用子查询删除记录						
2.5 索引和视图 ※※※						
2.5.1 基本表的定义、删除和修改						
1). 创建表						
CREATE TABLE 表名 (<列名> <数据类型> [DEFAULT <默认值>] [, ...])						
2) 修改表的结构						
1) 在一个表中增加一个新列						
在一表中增加一个新列的语句格式为：						
ALTER TABLE <表名> ADD <列名> <数据类型> [DEFAULT <默认值>]						
2) 修改一个表中已有的列						
ALTER TABLE <表名> MODIFY <列名> <数据类型> [DEFAULT <默认值>]						

3) 从一个表中删除一列

ALTER TABLE <表名>

DROP COLUMN <列名>

3) 截断表和删除表

1) 截断表

TRUNCATE TABLE <表名>

2) 删除表

DROP TABLE <表名>

2.5.2 索引的创建与删除

建立索引是加快查询速度的有效手段

◆建立索引

- DBMS 自动建立

- PRIMARY KEY

- UNIQUE

- DBA 或表的属主（即建立表的人）根据需要建立

◆维护索引

- DBMS 自动完成

◆使用索引

- DBMS 自动选择是否使用索引以及使用哪些索引

1). 创建索引

创建索引的语句格式如下：

CREATE INDEX <索引名> ON <表名>(<列名>[,<列名>].....)

2). 查看索引

SHOW INDEX FROM <表名>;

3). 删除索引

DROP INDEX <索引名> ON <表名>

2.5.3 视图

1. 为什么建立视图？

- 提供各种数据表现形式，隐藏数据的逻辑复杂性并简化查询语句；
- 提供某些安全性保证，简化用户权限的管理；
- 对重构数据库提供了一定的逻辑独立性。

视图（View）是由 SELECT 子查询语句定义的一个逻辑表，只有定义而无数据，是一个“虚表”。

1). 创建视图

CREATE [OR REPLACE] VIEW <视图名> [(<别名>[,<别名>]...)]

AS

<子查询语句>

[WITH CHECK OPTION [CONSTRAINT <约束名>]]

[WITH READ ONLY]

2) . 删除视图

DROP VIEW <视图名>

3). 修改视图

CREATE OR REPLACE VIEW 是用创建视图的语句将原来的视图覆盖掉。

使用 ALTER 语句是 MySQL 提供的另外一种修改视图的方法，

其语句格式如下：

ALTER VIEW 视图名[(别名[,别名]...)]

AS

SELECT 语句

[WITH CHECK OPTION];

4). 使用视图进行 DML 操作 简单视图： ① 数据是仅从一个表中提取； ② 不包含函数和分组数据； ③ 可以通过该视图进行 DML 操作。 复杂视图： ① 数据是从多个表中提取； ② 包含函数和分组数据； ③ 不一定能够通过该视图进行 DML 操作。 通过视图进行 DML 操作的规则： ① 可以在简单视图上执行 DML 操作； ② 如果在一个视图中包含了分组函数，或 GROUP BY 子句，或 DISTINCT 关键字，则不能通过该视图进行 DELETE、UPDATE、INSERT 操作； ③ 如果在一个视图中包含了由表达式组成的列，则不能通过该视图进行 UPDATE、INSERT 操作； ④ 如果在一个视图中包含引用表中为空的列，则不能通过该视图进行 INSERT 操作。 【课程小结】 本次课主要掌握索引的创建与删除，视图的创建、修改和删除，及对数据的增、删、改的操作。	2022 年国内十大信息泄露事件
--	-------------------------

《数据库原理与应用》 课程教案（第 7 次课 第五章）

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	0
教学单元（教学章、节或主题）：第5章 数据库的安全性					
目的、要求（分了解、理解、掌握三个层次） 1、了解实现数据库的安全性的方法和技术。 2、掌握数据库的安全性的设置方法。 3、熟悉数据的加密技术，统计数据库安全性。					
重点 1、数据库安全性控制 2、视图机制 3、MySQL 权限管理					
难点 数据库安全性控制					
教学步骤 签到 → 提问引起思考引入课程→ 理论讲解 → 小结与作业					
教具及教学手段 举例讲解、多媒体讲解、博思智慧学习平台教学工具相结合					
作业和思考题： 作业：教材第 5 章课后习题 思考题：数据库授权和回收权限的命令语句是什么？ 对用户账户进行管理的 SQL 命令？					
教 学 后 记					

由于任何系统的安全保护措施都不是完美无缺的，蓄意盗取、破坏数据的人总会想方设法打破控制。审计功能把用户对数据库的所有操作自动记录下来放入“审计日志”（audit log）中，称为审计跟踪。DBA 可以利用审计跟踪信息，重现导致数据库现有状况的一系列事件，找出非法存取数据的人、时间和内容等，为分析攻击者线索提供依据。一般地，将审计跟踪和数据库日志记录结合起来，会达到更好的安全审计效果。

5.5 数据加密

加密方法主要有两种，一种是替换方法，另一种是转换方法。

5.6 统计数据库的安全性

有一类数据库称为“统计数据库”，例如人口调查数据库，它包含大量的记录，但其目的只是向公众提供统计、汇总信息，而不是提供单个记录的内容。也就是查询的仅仅是某些记录的统计值，例如求记录数、和、平均值等。在统计数据库中，虽然不允许用户查询单个记录的信息，但是用户可以通过处理足够多的汇总信息来分析出单个记录的信息，这就给统计数据库的安全性带来严重的威胁。

5.7 MySQL 的安全设置※※※

5.7.1 用户管理

1. 创建用户账号

```
CREATE USER 用户 [IDENTIFIED BY [PASSWORD] '密码']  
[用户 [IDENTIFIED BY [PASSWORD] '密码']]...;
```

【说明】

① 用户的格式： 用户名@主机名

主机名指定了创建的用户使用 MySQL 连接的主机。“%”表示一组主机。
localhost 表示本地主机。

② IDENTIFIED BY 子句指定创建用户时的密码。如果密码是一个普通的字符串，就不需要使用 PASSWORD 关键字。

2. 修改用户密码

在创建用户后，允许对其进行修改，可以使用 SET PASSWORD 语句修改用户的登录密码，其语法形式如下：

```
SET PASSWORD FOR 用户='新密码';
```

3. 修改用户名

修改已存在的用户名，可以使用 RENAME USER 语句，其语法形式如下：
RENAME USER 旧用户名 TO 新用户名[,旧用户名 TO 新用户名][,...];

4. 删除用户账号

使用 DROP USER 语句可删除一个或多个 MySQL 用户，并取消其权限。
其语法形式如下：

```
DROP USER 用户[,...];
```

5.7.2 权限管理

1. 授予权限

平台操作演示
黑板讲解

(1) 授予 MySQL 字段级别权限 在 MySQL 中，使用 GRANT 语句授予权限。授予 MySQL 字段级别权限的语法形式如下： GRANT 权限名称(列名[,列名,...])[, 权限名称(列名[,列名,...]),...] ON TABLE 数据库名.表名或视图名 TO 用户[,用户,...] [WITH GRAND OPTION]; (2) 授予 MySQL 表级别权限 授予 MySQL 表级别权限的语法形式如下： GRANT 权限名称[, 权限名称,...] ON TABLE 数据库名.表名或视图名 TO 用户[,用户,...] [WITH GRAND OPTION]; (3) 授予 MySQL 存储程序级别权限 授予存储程序级别权限的语法形式如下： GRANT 权限名称[, 权限名称,...] ON FUNCTION PROCEDURE 数据库名.函数名 数据库名.存储过程名 TO 用户[,用户,...] [WITH GRAND OPTION]; (4) 授予 MySQL 数据库级别权限 授予 MySQL 数据库级别权限的语法形式如下： GRANT 权限名称[, 权限名称,...] ON 数据库名.* TO 用户[,用户,...] [WITH GRAND OPTION]; (5) 授予 MySQL 服务器管理员级别权限 授予 MySQL 服务器管理员级别权限的语法形式如下： GRANT 权限名称[, 权限名称,...] ON *.* TO 用户[,用户,...] [WITH GRAND OPTION]; 2. 撤销权限 (1) 撤销所有权限 撤销用户所有权限的 REVOKE 语句语法形式如下。 REVOKE ALL PRIVILEGES,GRANT OPTION FROM 用户[,用户,...]; (2) 撤销指定权限 撤销用户指定权限的 REVOKE 语句语法形式如下。 REVOKE 权限名称[(列名[,列名,...])][,权限名称[(列名[,列名,...])],...] ON *.* 数据库名.* 数据库名.表名或视图名 FROM 用户[,用户,...]; 5.7.3 角色管理 1. 创建角色 创建角色的语法形式如下。	
---	--

CREATE ROLE 角色; 角色格式: '角色名'@'主机名'。 2. 授予角色权限 授予角色权限的语法格式类似于授予用户权限, 只需将 GRANT 语句 TO 后的用户改为角色即可。 3. 授予用户角色 授予用户角色的语法形式如下。 GRANT 角色[,角色,...] TO 用户[,用户,...]; 4. 撤消用户角色 撤消用户角色的语法形式如下。 REVOKE 角色[,角色,...] FROM 用户[,用户,...]; 5. 删除角色 删除角色的语法形式如下。 DROP USER 角色[,角色,...]; 【课程小结】 本次课主要掌握安全控制策略, 及使用视图如何实现安全性控制。了解什么是审计跟踪, 统计数据库的安全性实现方法, 主要掌握 MySQL 下用户账号管理、权限管理及角色管理。	
--	--

《数据库原理与应用》 课程教案（第 8 次课 第 8 章）

授课方式 (请打√)	理论课 ☒ 讨论课 ☒ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	0
教学单元（教学章、节或主题）：第8章 使用实体-联系模型进行数据建模					
目的、要求（分了解、理解、掌握三个层次） 1、了解实体联系模型的相关概念 2、掌握局部 E-R 模型和全局 E-R 模型的设计过程和综合过程					
重点 1、局部 E-R 模型设计 2、全局 E-R 模型设计					
难点 如何将多个局部 E-R 模型综合成全局 E-R 模型					
教学步骤 签到 → 提问引起思考引入课程→ 理论讲解 → 小结与作业					
教具及教学手段 举例讲解、多媒体讲解、博思智慧学习平台教学工具相结合					
作业和思考题： 作业：教材第 8 章课后习题 思考题：局部 E-R 图合并时会产生哪些冲突？各冲突的解决方法？					
教 学 后 记					

多值属性（学士，硕士，博士）；一个零件可能有多种销售价格（经销、代销、批发、零售）。

（3）空值属性

当实体在某个属性上没有值时应使用空（NULL）值。

8.2.3 联系

1. 联系

概念：表示一个或多个实体间的关联关系。

2. 联系的设计

（1）二元联系

二元联系是指两个实体之间的联系，这种联系比较常见。

（2）一元联系

一元联系是指一个联系所关联的是同一个实体集中的两个实体。这种情况比较特殊，但在现实生活中也是存在的，有时，也称这种联系为递归联系。

（3）三元联系

三元联系是指三个实体间的联系，这种联系也比较常见。

8.3 利用 E-R 模型的数据库概念设计

8.3.1 局部 E-R 模型设计

设计局部 E-R 模型，关键是确定：

（1）一个概念是用实体还是属性表示？

（2）一个概念是作实体的属性还是联系的属性？

1. 实体和属性的数据抽象※※※

（1）分类

定义某一类概念作为现实世界中一组对象的类型，将一组具有某些共同特性和行为的对象抽象为一个实体。

（2）聚集

定义某个类型的组成成分。将对象的组成成分抽象为实体的属性。

2. 实体和属性的取舍

实体和属性是相对而言的，往往要根据实际情况进行必要的调整，在调整时要遵守两条原则：

（1）属性不能再具有需要描述的性质，即属性必须是不可分的数据项，不能再由另一些属性组成。

（2）属性不能与其它实体具有联系，联系只发生在实体之间。

3. 属性在实体与联系间的分配

当多个实体用到同一属性时，将导致数据冗余，从而可能影响存储效率和完整性约束，因而需要确定把它分配给哪个实体。一般把属性分配给那些使用频率最高的实体，或分配给实体值少的实体。例如，“课名”属性，不需要在“学生”和“课程”实体中都出现，一般将其分配给“课程”实体作属性。

4. 局部 E-R 模型设计过程

1) 确定局部结构范围

2) 实体定义

3) 联系定义

4) 属性分配

8.3.2 全局 E-R 模型设计※※※

局部 E-R 模型的合并

对于各局部 E-R 模型，需要合并成一个整体的全局 E-R 模型。一般来说，合并可以有两种方式：

- 多个局部 E-R 图一次合并，如图 8-16 (a) 所示。
- 逐步合并，用累加的方式一次合并两个局部 E-R 图，如图 8-16 (b) 所示。

消除冲突

1) 属性冲突

属性冲突又包括属性域冲突和属性取值单位冲突。

(1) 属性域冲突。即属性值的类型、取值范围或取值集合不同。

(2) 属性取值单位冲突。

2) 命名冲突

(1) 同名异义。即不同意义的对象在不同的局部应用中具有相同的名字。

(2) 异名同义。即同一意义的对象在不同的局部应用中具有不同的名字。

3) 结构冲突

(1) 同一对象在不同应用中具有不同的抽象。

解决方法：通常是把属性变换为实体或实体变换为属性，使同一对象具有相同的抽象。

(2) 同一实体在不同局部 E-R 图中所包含的属性个数和属性排列次序不完全相同。

解决方法：使该实体的属性取各局部 E-R 图中属性的并集，再适当调整属性的次序。

(3) 实体间的联系在不同的局部 E-R 图中为不同类型。

解决方法：根据应用的语义对实体联系的类型进行综合或调整。

全局 E-R 模型的优化

(1) 实体的合并

这里的合并是指相关实体的合并。在信息检索时，涉及到多个实体的信息要通过连接操作获得。因而减少实体个数，可减少连接的开销，提高处理效率。

一般在权衡利弊后，可以把 1:1 联系的两个实体合并。

(2) 冗余属性的消除

通常在各个局部结构中是不允许冗余属性存在的，但在综合成全局 E-R 模型后，可能产生全局范围内的冗余属性。

(3) 冗余联系的消除

在全局模型中可能存在有冗余的联系，通常利用规范化理论中函数依赖的概念消除冗余。

【课程小结】

E-R 图是数据库设计的关键，如何将多个局部 E-R 图综合成全局 E-R 图，这个过程需多作题熟练掌握。

《数据库原理与应用》课程教案(第 9 次课 9.1,9.2)

编制日期： 2021 年 3 月 3 日

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	课时 安排	2
教学单元(教学章、节或主题)：第9章 关系模型规范化设计理论(第1节+第2节)			
目的、要求(分了解、熟悉、掌握三个层次)： 1、掌握关系中存在的异常及其原因 2、掌握数值依赖的公理系统(闭包；最小函数依赖集)			
重点： 1、关系模式中存在异常的判断 2、求关系模式的函数依赖集			
难点： 求最小函数依赖集			
教学步骤： 提问引起思考引入课程 精讲课堂内容 课堂小结 布置作业			
教具及教学手段：多媒体讲解、举例讲解			
作业和思考题： 练习： 教材第9章课后习题 思考题： 不好的关系模式会存在哪些异常情况？各异常情况导致的原因？			

教 学 内 容 (含时间分配)	板书或旁注
-----------------	-------

教学环节	复习提问	新课讲解	课堂实践	每课小结	布置作业
时间分配	5	68	10	5	2

第9章 关系模型规范化设计理论

【课程导入】

问题：给出的任何一个关系都是合理的吗？怎么判断？如何将关系分解为合理的关系？

【讲授内容】

9.1 关系模式中可能存在的异常※※※

1. 数据冗余

2. 插入异常：插入异常的表现是，元组插入不进去。

3. 删除异常：删除异常的表现是，删除时，删掉了其他不应删除的信息。

4. 更新异常：更新异常的表现是，修改一个元组，却要求修改多个元组。

关系模式异常现象产生的原因，就是由于关系模式中存在的这些复杂的数据依赖关系所导致的。在设计关系模式时，如果将各种有联系的实体数据集中于一个关系模式中，不仅造成关系模式结构冗余、包含的语义过多，也使得其中的数据依赖变得错综复杂，不可避免地要违背以上某个或多个限制，从而产生异常。

9.2 函数依赖

9.2.1 函数依赖定义

定义 9.1 设 $R(U)$ 是属性集 U 上的关系模式， X 和 Y 是 U 的子集。若对于 $R(U)$ 的任意一个可能的关系 r ，对于 X 的每一个具体值， Y 都有唯一的具体的值与之对应，则称 X 函数决定 Y ，或 Y 函数依赖于 X ，记作 $X \rightarrow Y$ 。称 X 为决定因素， Y 为依赖因素。

9.2.2 发现函数依赖

1. 根据完整的样本数据发现函数依赖

这种方法是根据完整的样本数据和函数依赖的定义来实现的。在没有样本数据或者只有部分样本数据时，则不能用此方法确定数据之间函数依赖的关系。

2. 根据数据语义发现函数依赖

(1) 如果 X 和 Y 之间是 1: 1 的联系，则存在的函数 $X \rightarrow Y$ 和 $Y \rightarrow X$ ；

(2) 如果 X 和 Y 之间是 1: n 的联系，则存在的函数 $Y \rightarrow X$ ；

(3) 如果 X 和 Y 之间是 m : n 的联系，则 X 和 Y 之间不存在函数依赖关系。

9.2.3 最小函数依赖集

1. 函数依赖的推理规则

函数依赖的推理规则如下：

(1) A1（自反性）：如果 $Y \subseteq X \subseteq U$ ，则 $X \rightarrow Y$ 。

(2) A2（增广性）：如果 $X \rightarrow Y$ 且 $Z \subseteq U$ ，则 $XZ \rightarrow YZ$ 。

(3) A3（传递性）：如果 $X \rightarrow Y$ 且 $Y \rightarrow Z$ ，则 $X \rightarrow Z$ 。

(4) B1（合并性）：如果 $X \rightarrow Y$ 且 $X \rightarrow Z$ ，则 $X \rightarrow YZ$ 。

(5) B2（分解性）：如果 $X \rightarrow YZ$ ，则 $X \rightarrow Y$ 、 $X \rightarrow Z$ 。

(6) B3（结合性）：如果 $X \rightarrow Y$ 且 $W \rightarrow Z$ ，则 $XW \rightarrow YZ$ 。

(7) B4（伪传递性）：如果 $X \rightarrow Y$ 且 $WY \rightarrow Z$ ，则 $XW \rightarrow Z$ 。

A1~A3 就是有名的 Armstrong 公理，B1~B4 是 Armstrong 公理的推论。

2. 求最小函数依赖集※※※

如果函数依赖集 F 满足下列条件，则称 F 为一个最小函数依赖集。

(1) 每个函数依赖的右边都是单属性（可以通过 B2 分解性实现）；

(2) 函数依赖集 F 中没有冗余的函数依赖（即 F 中不存在这样的函数依赖

黑板例题推导讲解

$X \rightarrow Y$ ，使得 F 与 $F - \{X \rightarrow Y\}$ 等价)；

(3) F 中每个函数依赖的左边没有多余的属性 (即 F 中不存在这样的函数依赖 $X \rightarrow Y$ ， X 有真子集 W 使得 $F - \{X \rightarrow Y\} \cup \{W \rightarrow Y\}$ 与 F 等价)。

显然，每个函数依赖集至少存在一个最小依赖集，但并不一定唯一。

【例 9-4】设 F 是关系模式 $R(A, B, C)$ 的 FD 集， $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$ ，试求最小函数依赖集。

解答：

(1) 先把 F 中的函数依赖写成右边是单属性形式：

$$F = \{A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$$

删去一个 $A \rightarrow B$ ，得：

$$F = \{A \rightarrow B, A \rightarrow C, B \rightarrow C, AB \rightarrow C\}$$

(2) 删去冗余的函数依赖。 F 中 $A \rightarrow C$ 可从 $A \rightarrow B$ 和 $B \rightarrow C$ 推出，因此 $A \rightarrow C$ 是冗余的，删去，得：

$$F = \{A \rightarrow B, B \rightarrow C, AB \rightarrow C\}$$

(3) 消除函数依赖左边冗余的属性。 F 中的 $AB \rightarrow C$ ，因为有 $B \rightarrow C$ ，所以 A 多余，删去，得到最小函数依赖集为：

$$F = \{A \rightarrow B, B \rightarrow C\}$$

【例 9-5】设关系模式 $R(A, B, C, D, E, G, H)$ 上的函数依赖集 $F = \{AC \rightarrow BEGH, A \rightarrow B, C \rightarrow DEH, E \rightarrow H\}$ ，求 F 的最小函数依赖集。

解答：

(1) 把每个 FD 的右边拆成单属性，得到 9 个 FD，得：

$$F = \{AC \rightarrow B, AC \rightarrow E, AC \rightarrow G, AC \rightarrow H, A \rightarrow B, C \rightarrow D, C \rightarrow E, C \rightarrow H, E \rightarrow H\}$$

(2) 消除冗余的 FD，得：

$$F = \{AC \rightarrow B, AC \rightarrow E, AC \rightarrow G, AC \rightarrow H, A \rightarrow B, C \rightarrow D, C \rightarrow E, E \rightarrow H\}$$

(3) 消除 FD 中左边冗余的属性。因为 $A \rightarrow B$ ，所以消去 $AC \rightarrow B$ 中的 C ；因为 $C \rightarrow E$ ，所以消去 $AC \rightarrow E$ 的 A ；因为由 $C \rightarrow E$ 、 $E \rightarrow H$ ，可推出 $C \rightarrow H$ ，所以消去 $AC \rightarrow H$ 中的 A ，得 $C \rightarrow H$ ，因为可由 $C \rightarrow E$ 、 $E \rightarrow H$ 推出，所以将 $AC \rightarrow H$ 删去，得到的 F 为：

$$F = \{A \rightarrow B, C \rightarrow E, AC \rightarrow G, A \rightarrow B, C \rightarrow D, C \rightarrow E, E \rightarrow H\}$$

精简后，得

$$F = \{A \rightarrow B, C \rightarrow E, AC \rightarrow G, C \rightarrow D, E \rightarrow H\}$$

(4) 再把左边相同的 FD 合并起来，得到最小的函数依赖集为：

$$F = \{A \rightarrow B, C \rightarrow DE, AC \rightarrow G, E \rightarrow H\}$$

【课程小结】

函数依赖，码，范式概念不太好理解，通过示例来讲解这些抽象的概念。为规范化的理解打好基础。闭包相对于规范化的概念还是要好理解的。

《数据库原理与应用》课程教案(第 10 次课 9.3)

编制日期： 2021 年 3 月 3 日

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	课时 安排	2
教学单元（教学章、节或主题）：第9章 关系模型规范化设计理论（第3节）			
目的、要求（分了解、熟悉、掌握三个层次）： 1、掌握通过最小函数依赖集求候选码 2、掌握范式的规范化（2NF；3NF；BCNF）。			
重点： 1、求关系的候选键 2、根据候选键对关系模式的规范化			
难点： 求候选键、规范化			
教学步骤： 提问引起思考引入课程 精讲课堂内容 课堂小结 布置作业			
教具及教学手段：多媒体讲解、举例讲解			

1NF、2NF、3NF 的判断方法？

[illegible]

(1) 查看函数依赖集 F 中的每个形如 $X_i \rightarrow Y_i$ ($i=1, \dots, n$) 的函数依赖关系。看哪些属性在所有 Y_i ($i=1, \dots, n$) 中一次也没有出现过, 设没有出现过的属性集为 P ($P=U-Y_1-Y_2-\dots-Y_n$)。则当 $P=\emptyset$ 时, 转步骤 (4); $P \neq \emptyset$ 时, 转步骤 (2)。

(2) 根据候选键的定义, 候选键中应必包含 P (因为没有其它属性能决定 P , 但自己能决定自己)。考察 P , 如果 P 满足候选键定义, 则 P 为候选键, 并且候选键只有 P 一个, 然后转步骤 (5) 结束; 如果 P 不满足候选键定义, 则转步骤 (3) 继续。

(3) P 可以分别与 $\{U-P\}$ 中的每一个属性合并, 合成 $P_1, P_2, \dots, P_m$ 。再分别判断 P_j ($j=1, \dots, m$) 是否满足候选键定义, 能成立则找到了一个候选键, 没有则放弃。合并一个属性如果不能找到或不能找全候选键, 可进一步考虑 P 与 $\{U-P\}$ 中的 2 个 (或 3 个, 4 个, ...) 属性的所有组合分别进行合并, 继续判断分别合并后的各属性组是否满足候选键的定义, 如此下去, 直到找出 R 的所有候选键为止。转步骤 (5) 结束。

注意: 如果属性组 K 已有 $K \rightarrow U$, 则不需要再去考察含 K 的其它属性组合, 显然它们都不可能再是候选键了 (根据候选键定义的第②项)。

(4) 如果 $P=\emptyset$, 则可以先考察 $X_i \rightarrow Y_i$ ($i=1, \dots, n$) 中的单个 X_i , 判断 X_i 是否满足候选键定义。如果成立则 X_i 为候选键。剩下不是候选键的, 可以考察它们两个或多个的组合, 查看这些组合是否满足候选键定义, 从而找出其它可能还有的候选键。转步骤 (5) 结束。

(5) 本方法结束。

【课程小结】

规范化的概念比较难建立, 也比较难懂, 必须多举些实例由浅入深的讲解。要求学生课后多看例题, 多琢磨。

$X \twoheadrightarrow Y$

--	--

《数据库原理与应用》 课程教案(第 11 次课 9.4)

编制日期： 2021 年 3 月 3 日

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	课时 安排	2
---------------	--	----------	---

教学单元（教学章、节或主题）：第9章 关系模型规范化设计理论（第4节）	
目的、要求（分了解、熟悉、掌握三个层次）： 3、掌握通过最小函数依赖集求候选码 4、掌握范式的规范化（2NF；3NF；BCNF）。	
重点： 1、求关系的候选键 2、根据候选键对关系模式的规范化	
难点： 求候选键、规范化	
教学步骤： 提问引起思考引入课程 精讲课堂内容 课堂小结 布置作业	
教具及教学手段：多媒体讲解、举例讲解	
作业和思考题： 练习： 教材第9章课后习题 思考题： 1NF、2NF、3NF 的判断方法？	

教 学 内 容（含时间分配）	板书或旁注
----------------	-------

教学环节	复习提问	新课讲解	课堂实践	每课小结	布置作业
时间分配	5	68	10	5	2

第9章 关系模型规范化设计理论

【课程导入】

问题：对不合理的关系模式怎么分解才能变成合理的？

【讲授内容】

9.4 关系模式的规范化※※※

9.4.1 范式及规范化

1. 范式

范式（Normal Form, NF）是指关系模式的规范形式。

关系模式上的范式有6个：1NF（称作第一范式，以下类同）、2NF、3NF、BCNF、4NF、5NF。各范式间的联系为：

$$5NF \subset 4NF \subset BCNF \subset 3NF \subset 2NF \subset 1NF$$

2. 规范化

将一个给定的关系模式转化为某种范式的过程，称为关系模式的规范化过程。

规范化一般采用分解的办法，将低级别范式向高级别范式转化，使关系的语义单纯化。

规范化的目的是逐渐消除异常。

9.4.2 完全函数依赖、部分函数依赖和传递函数依赖

1. 完全函数依赖和部分函数依赖

定义 9.4 设 R 是一个具有属性集合 U 的关系模式，X 和 Y 是 U 的子集。

如果 $X \rightarrow Y$ ，并且对于 X 的任何一个真子集 Z， $Z \rightarrow Y$ 都不成立，则称 Y 完全函数依赖于 X，记作 $X \twoheadrightarrow Y$ ；

如果 $X \rightarrow Y$ ，并且对于 X 的任何一个真子集 Z， $Z \rightarrow Y$ 都成立，则称 Y 部分函数依赖于 X，记作 $X \rightharpoonup Y$ 。

2. 传递函数依赖

定义 9.5 设 R 是一个具有属性集合 U 的关系模式，X、Y、Z 是 U 的子集，且 X、Y、Z 是不同的属性集。如果 $X \rightarrow Y$ ， $Y \rightarrow X$ 不成立， $Y \rightarrow Z$ ，则称 Z 传递函数依赖于 X，记作 $X \twoheadrightarrow Z$ 。

说明：

- （1）如果 $X \rightarrow Y$ ，且 $Y \rightarrow X$ ，则称 X 与 Y 等价，记作 $X \leftrightarrow Y$ 。
- （2）如果定义中 $Y \rightarrow X$ 成立，则 X 与 Y 等价，这时称 Z 对 X 直接函数依赖，而不是传递函数。

9.4.3 以函数依赖为基础的范式

1. 第一范式（1NF）

定义 9.6 设 R 是一个关系模式。如果 R 中每个属性的值域，都是不可分的原子值，则称 R 是第一范式，记作 1NF。

1NF 是关系模式具备的最起码的条件。

2. 第二范式（2NF）

定义 9.7 如果关系模式 R 是 1NF，而且 R 中所有非主属性都完全函数依赖于任意一个候选键，则称 R 是第二范式，记作 2NF。

2NF 的实质是不存在非主属性“部分函数依赖”于候选键的情况。

非 2NF 关系或 1NF 关系向 2NF 的转换原则是消除其中的部分函数依赖，一般是将一个关系模式分解成多个 2NF 的关系模式，即将部分函数依赖于候选键的非主属性及其决定属性移出，另成一个关系，使其满足 2NF。可以总结为如下方法：

3. 第三范式 (3NF)

定义 9.8 如果关系模式 R 是 2NF, 而且 R 中所有非主属性对任何候选键都不存在传递函数依赖, 则称 R 是第三范式, 记作 3NF。

3NF 是从 1NF 消除非主属性对候选键的部分函数依赖, 和从 2NF 消除传递函数依赖而得到的关系模式。

2NF 关系向 3NF 转换的原则是消除传递函数依赖, 将 2NF 关系分解成多个 3NF 关系模式。可以总结为如下方法:

设关系模式 R 属性集合为 U, 主键是 W, R 上还存在函数依赖 $X \rightarrow Z$, 并且 Z 是非主属性, Z 不包含于 X, X 不是候选键, 这样 $W \rightarrow Z$ 就是一个传递依赖。此时应把 R 分解成两个关系模式:

- R1 (XZ), 主键是 X;
- R2 (Y), 其中 $Y=U-Z$, 主键仍是 W, 外键是 X。

4. Boyce-Codd 范式 (BCNF)

定义 9.9 如果关系模式 R 是 1NF, 且对于 R 中每个函数依赖 $X \rightarrow Y$, X 必为候选键, 则称 R 是 BCNF 范式。

由 BCNF 的定义可以知, 每个 BCNF 范式应具有以下 3 个性质:

- (1) 所有非主属性都完全函数依赖于每个候选键;
- (2) 所有主属性都完全函数依赖于每个不包含它的候选键;
- (3) 没有任何属性完全函数依赖于非键的任何一组属性。

3NF 关系向 BCNF 转换的原则是消除主属性对候选键的部分和传递函数依赖, 将 3NF 关系分解成多个 BCNF 关系模式。

9.4.4 关系的分解

1. 无损连接分解※※※

如果关系模式 R 上的任一关系 r 都是它在各分解模式上投影的自然连接(自然连接是一种特殊的等值连接, 结果中去掉重复的属性列), 则该分解就是无损连接分解, 也称无损分解。否则就是有损连接分解, 或称有损分解。

2. 无损连接分解的测试

算法 9.2 无损分解的测试方法。

输入: 关系模式 $R=(A_1, A_2, \dots, A_n)$, F 是 R 上成立的函数依赖集, $p=\{R_1, R_2, \dots, R_k\}$ 是 R 的一个分解。

输出: 确定 p 是否为 R 的无损分解。

步骤:

(1) 构造一张 k 行 n 列的表格, 每列对应一个属性 A_j ($1 \leq j \leq n$), 每行对应一个模式 R_i ($1 \leq i \leq k$)。如果 A_j 在 R_i 中, 那么表格的第 i 行第 j 列处填上符号 a_j , 否则填上 b_{ij} (a_j, b_{ij} 仅是一种符号, 无专门含义)。

(2) 把表格看成模式 R 的一个关系, 反复检查 F 中每个函数依赖在表格中是否成立, 若不成立, 则修改表格中的值。修改方法如下:

对于 F 中的一个函数依赖 $X \rightarrow Y$, 在表格中寻找对应于 X 中属性的所有列上符号 a_i 或 b_{ij} 全相同的那些行, 按下列情况处理:

① 如果表格中有两行 (或多行) 这样的行, 则让这些行中对应于 Y 中属性的所有列的符号相同: 如果符号中有一个 a_j , 那么其它全都改成 a_j ; 如果没有 a_j , 那么用其中一个 b_{ij} 替换其它值 (尽量把下标 i, j 改成较小的数)。

② 如果没有找到两个这样的行, 则不用修改。

对 F 集中所有函数依赖重复执行步骤 2, 直到表格不能修改为止。

(3) 若修改的最后一张表格中有一行是全 a, 即 $a_1, a_2, \dots, a_n$, 那么称 p 相对于 F 是无损分解, 否则称有损分解。

【课程小结】

规范化的概念比较难建立, 也比较难懂, 必须多举些实例由浅入深的讲解。要求学生课后多看例题, 多琢磨。

X --> Y

《数据库原理与应用》课程教案(第 12 次课 10.1~10.3)

编制日期： 2021 年 3 月 3 日

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	课时 安排	2
教学单元 (教学章、节或主题)： 第10章 数据库设计第1~3节			
目的、要求 (分了解、熟悉、掌握三个层次)： 1、掌握概念结构设计(概念结构的概念；概念结构设计的方法和步骤 2、熟悉数据抽象与局部视图设计；视图集成) 3、掌握逻辑结构设计 (E-R 图向关系模型的转换；数据模型的优化；设计用户子模式)。			
重点： 1、数据库设计方法与基本步骤(概念结构设计，逻辑结构设计)。 2、数据库设计方法与基本步骤。			
难点： 概念结构设计，E-R 图向关系模型的转换，数据模型的优化。			
教学步骤： 提问引起思考引入课程 精讲课堂内容 课堂小结 布置作业			
教具及教学手段：多媒体讲解、举例讲解			
作业和思考题： 练习： 教材第 10 章课后习题 思考题： 概念模型转化为关系模型的方法？			

行分析；然后写出可行性分析报告；组织专家进行讨论其可行性。

③ 确定数据库系统的总目标和制定项目开发计划。在得到决策部门批准后，就正式进入数据库系统的开发工作。

2. 需求分析阶段

需求分析是整个设计过程的基础，是最困难、最耗费时间的一步

3. 设计阶段

(1) 概念设计阶段

(2) 逻辑设计阶段

(3) 物理设计阶段

4. 实现阶段

① 用 DBMS 提供的 DDL（数据定义语言）定义数据库结构；

② 组织数据入库；

③ 编制与调试应用程序。

5. 测试阶段

在这一阶段，对数据库的结构及使用进行测试；对数据库的并发控制、恢复、安全性、完整性措施进行测试。采用软件工程的白盒测试和黑盒测试方法，对应用程序进行单元测试和集成测试。

6. 运行维护阶段

10.2 需求分析

10.2.1 需求分析的任务

1. 信息需求

2. 处理需求

3. 运行需求

10.2.2 需求分析的过程

1. 分析用户活动，产生业务流程图。

2. 确定系统范围，产生系统关联图。

3. 分析用户活动涉及的数据，产生数据流图。

4. 分析系统数据，产生数据字典。

10.2.3 用户需求调研的方法

1. 审阅以前的研究及应用情况。

2. 查阅文档。

3. 发调查问卷。

4. 同用户交谈。

5. 现场调查。

10.3 概念数据建模

10.3.1 建模方法 E-R 建模方法设计概念模型一般有两种方法。 (1) 视图集成建模法。 (2) 集中模式建模法。 10.3.2 建模的基本任务与步骤 1. 用户视图建模 首先，构造实体。构造方法如下： (1) 根据数据流图和数据字典提供的情况，将一些对应于客观事物的数据项汇集、形成一个实体，数据项则是该实体的属性。这里的事物可以是具体的事物或抽象的概念、事物联系或某一事件等。 (2) 将剩下的数据项用一对多的分析方法，再确定出一批实体。某数据项若与其它多个数据项之间存在一对多的对应关系，那么这个数据项就可以作为一个实体，而其他多个数据项则作为它的属性。 (3) 分析最后一些数据项之间的紧密程度，又可以确定一批实体。如果某些数据项完全依赖于另一些数据项，那么所有这些数据项可以作为一个实体，而后者“另一些数据项”可以作为此实体的键。 2. 视图集成 视图集成要解决如下问题。 (1) 命名冲突。指属性、联系、实体的命名存在冲突，冲突有同名异义和同义异名两种。 (2) 结构冲突。同一概念在一个视图中可作为实体，在另一个视图中可作为属性或联系。 (3) 属性冲突。相同的属性在不同的视图中有不同的取值范围。例如，学号在一个视图中可能是字符串，在另一个视图中可能是整数。有些属性采用不同的度量单位。例如，身高在一个视图中用厘米作单位，在另一个视图中可能用米作单位。 (4) 标识的不同。要解决多标识机制。例如，在一个视图中，可能用学号唯一标识学生，而在另外的一些视图中，可能用校园卡卡号作为学生的唯一标识。 (5) 区别数据的不同子集。例如，学生可分为本科生、硕士生、博士生。 【课程小结】 数据库设计篇总体来说不太难，要总结和理清我们的处理问题的思路。多讲些工程实例，使学生掌握这种设计思想，对以后的项目知道如何下手。	
--	--

《数据库原理与应用》课程教案(第 13 次课 10.4~10.7)

编制日期： 2021 年 3 月 3 日

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	课时 安排	2
教学单元（教学章、节或主题）： 第10章 数据库设计第4~7节			
目的、要求（分了解、熟悉、掌握三个层次）： （1）逻辑结构设计（将 E-R 图转换为关系模型） （2）物理结构设计（索引存取方法、不适于建立索引的情况） （3）数据库的实现与测试 （4）数据库的运行与维护 教学要求： （1）掌握如何将概念模型转换为关系模型； （2）了解物理设计中索引的使用； （3）了解数据库实现过程及运行维护的工作。			
重点： 1、数据库设计方法与基本步骤(概念结构设计，逻辑结构设计)。 2、数据库设计方法与基本步骤。			
难点： 概念结构设计，E-R 图向关系模型的转换，数据模型的优化。			
教学步骤： 提问引起思考引入课程 精讲课堂内容 课堂小结 布置作业			
教具及教学手段：多媒体讲解、举例讲解			

概念模型转化为关系模型的方法？

教 学 内 容 (含时间分配)						板书或旁注
教学环节	复习提问	新课讲解	课堂实践	每课小结	布置作业	
时间分配	5	68	10	5	2	

第 10 章 数据库设计

【课程导入】

问题：你作为一个数据库设计人员，如何向客户了解所作系统的功能，并从中分析出用到的数据？

【讲授内容】

10.4 逻辑结构设计※※※

10.4.1 E-R 图向关系模型的转换

1. 实体转换成关系模式

实体转换成关系模式很直接，实体的名称即是关系模式的名称，实体的属性则为关系模式的属性，实体的主键就是关系模式的主键。

2. 联系的转换

1) 二元联系的转换

(1) 1: 1 联系的转换

一个 1: 1 联系可以转换为一个独立的关系，也可以与任意一端对应的关系模式合并。

① 转换为一个独立的关系模式，则与该联系相连接的各实体的键及联系本身的属性均转换为该关系模式的属性，每个实体的键均是该关系模式的候选键。

② 与某一端实体对应的关系模式合并，则需要在该关系模式的属性中加入另一个关系模式的键（作为外键）和联系本身的属性。

(2) 1: n 联系的转换

在 n 端实体转换的关系模式中加入 1 端实体的键（作为外键）和联系的属性。

(3) m: n 联系的转换

将联系转换为一个独立的关系模式，其属性为两端实体的键（作为外键）加上联系的属性，两端实体的键组成该关系模式的键或键的一部分。

10.4.2 采用 E-R 模型的逻辑设计步骤 (1) 导出初始关系模型 逻辑设计的第 1 步是把概念设计的结果，即全局 E-R 模型，转换成初始关系模型。 (2) 规范化处理 对于从 E-R 图转换来的关系模式，就要以关系数据库规范化设计理论为指导，对得到的关系模式逐一分析，确定它们分别是第几范式，并通过必要的分解来得到一组 3NF 的关系。 (3) 模式评价 模式评价的目的是检查已给出的数据库模式是否完全满足用户的功能要求，是否具有较高的效率，并确定需要加以修正的部分。模式评价主要包括功能和性能两个方面。 (4) 模式优化 根据模式评价的结果，对已生成的模式集进行优化。在后续的内容中将重点讲解优化的方法。 在逻辑设计阶段，还要设计出全部外模式。外模式是面向各个最终用户的局部逻辑结构。外模式体现了各个用户对数据库的不同观点，也提供了某种程度的安全控制。 【课程小结】 数据库设计篇总体来说不太难，要总结和理清我们的处理问题的思路。多讲些工程实例，使学生掌握这种设计思想，对以后的项目知道如何下手。	
---	--

--	--

《数据库原理与应用》 课程教案（第 14 次课 第 6 章）

授课方式 (请打√)	理论课 ☒ 讨论课 ☒ 实践课 ☐ 习题课 ☐ 其他 ☐	学时安排			
		课 内	2	课 外	0
教学单元（教学章、节或主题）： 第6章 事务与并发控制					
目的、要求（分了解、理解、掌握三个层次） 1.了解事务的基本概念 2.熟悉事务的四个特性 3.掌握并发控制会引发的问题。 4、熟悉封锁 5、掌握封锁协议 6、掌握 MySQL 的并发控制 授课计划表要求 （1）事务的特性及 MySQL 事务控制的实现 （2）数据库并发控制的基本概念。 （3）封锁协议。 （4）活锁和死锁。 （5）并发调度的可串行性。					

重点	
1、事务概念 2、ACID 特性 3、并发控制引发的问题 1、封锁协议 2、封锁协议带来的问题 3、二级封锁协议	
难点	
判断并发控制问题类型 1、使用封锁协议解决并发问题 2、灵活应用 MySQL 下的并发控制语句	
教学步骤	
签到 → 提问引起思考引入课程→ 理论讲解 → 小结与作业	
教具及教学手段	
举例讲解、多媒体讲解、博思智慧学习平台教学工具相结合	
作业和思考题：	
作业：教材第 6 章课后习题 思考题： 事务的四个特性及其特点？ 三级封锁协议分别解决并发控制带来的哪些问题？	
教 学 后 记	

教 学 过 程						板 书 或 旁 注
教学环节	复习提问	新课讲解	课堂实践	每课小结	布置作业	
时间分配	5	68	10	5	2	
第 6 章 事务与并发控制						

【课程导入】

问题：在银行转账的业务中，出现当钱从一个卡中转出去，但因为某种突出原因，转出的钱没有进入到指定的卡中，该如何解决？

问题：2 个事务并发的购买火车票，怎样防止一张票卖给 2 个人的呢？

【讲授内容】

6.1 事 务

（1）用户能将每一事务的执行当作是“原子”的。

（2）在多个事务并发执行的情况下，每个事务都是各自独立的，它既不干涉别的事务，也不受到别的事务的干涉，这称为事务的“隔离性”。

（3）隔离执行事务时，必须保证数据库中的数据，在操作前和操作后是一致的，这称为事务的“一致性”。

（4）事务成功地完成后，其结果（尤其是对数据库的变更）状态是永久的，这称为事务的“持久性”。

6.2 事务的 ACID 特性***

6.2.1 原子性

事务的原子性是指事务中包含的所有操作要么全做，要么全不做。也就是说，事务的所有活动在数据库要么全部反映，要么全不反映，以保证数据库是一致的。

6.2.2 一致性

一致性是指数据库在事务操作前和事务处理后，其中的数据必须都满足业务规则约束。

6.2.3 隔离性

隔离性是数据库允许多个并发事务同时对其中的数据进行读写和修改的能力，隔离性可以防止多个事务并发执行时，由于它们的操作命令交叉执行而导致的数据不一致状态。

6.2.4 持久性

事务的持久性表示为：当事务处理结束后，它对数据的修改应该是永久的，即使是系统在遇到故障的情况下也不会丢失。

6.2.5 MySQL 事务控制语句

应用程序主要通过指定事务启动和结束的时间来控制事务。

1. 事务模式

MySQL 有 3 种事务模式：自动提交事务模式、显式事务模式和隐性事务模式。

（1）自动提交事务模式。每条单独的语句都是一个事务，是 MySQL 默认的事务管理模式。在此模式下，当一条语句成功执行后，它被自动提交（系统变量 AUTOCOMMIT 值为 1），而当它执行过程中产生错误时，则被自动回滚。

(2) 显式事务模式。该模式允许用户定义事务的启动和结束。事务以 BEGIN WORK 或 START TRANSACTION 语句显式开始，以 COMMIT 或 ROLLBACK 语句显式结束。

(3) 隐性事务模式。在当前事务完成提交或回滚后，新事务自动启动。隐性事务不需要使用 BEGIN WORK 或 START TRANSACTION 语句标识事务的开始，但需要以 COMMIT 或 ROLLBACK 语句来提交或回滚事务。执行 SET @@AUTOCOMMIT=0;语句可以使 MySQL 进入隐性事务模式。

2. 开始事务

MySQL 默认设置，事务都是自动提交的，即执行 SQL 语句后会马上执行 COMMIT 操作。因此要显式地开启一个事务必须使用 START TRANSACTION 或 BEGIN WORK 语句，其语法形式如下：

START TRANSACTION;

或

BEGIN WORK;

【说明】

在存储过程中只能使用 START TRANSACTION 语句来开启一个事务，因为 MySQL 数据库分析器会自动将 BEGIN 识别为 BEGIN...END 语句。

3. 提交事务

COMMIT 语句用于结束一个用户定义的事务，保证对数据的修改已经成功地写入数据库，此时事务正常结束。其语法形式如下：

COMMIT [WORK] [AND [NO] CHAIN] [[NO] RELEASE];

【说明】

① 提交事务的最简单形式，只需发出 COMMIT 命令，详细的写法是 COMMIT WORK。

② AND CHAIN 子句会在当前事务结束时，立刻启动一个新事务，并且新事务与刚结束的事务有相同的隔离等级。

③ RELEASE 子句在终止了当前事务后，会让服务器断开与当前客户端的连接。

④ NO 关键字可以抑制 CHAIN 或 RELEASE 完成。

4. 回滚事务

回滚事务使用 ROLLBACK 语句，回滚会结束用户的事务，并撤消正在进行的所有未提交的修改（即 BEGIN WORK 或 START TRANSACTION 后的所有修改）。ROLLBACK 语句的语法形式如下：

ROLLBACK [WORK] [AND [NO] CHAIN] [[NO] RELEASE];

6.3 并发控制***

6.3.2 并发执行可能引起的问题

1. 丢失更新 (lost update)

丢失更新又称为覆盖未提交的数据，也就是说，一事务更新的数据尚未提交，而另一事务又将该未提交的更新数据再次更新，使得前一事务更新数据丢失。

原因：由于两个（或多个）事务对同一数据并发地写入引起，称为写—写冲突。

结果：与串行地执行两个（或多个）事务的结果不一致。

结合例子仔细讲解
结合练习学生掌握

2. 不可重复读（unrepeatable read）

不可重复读也称为读值不可复现。由于另一事务对同一数据的写入，一个事务对该数据两次读到的值不一样。

原因：该问题因读—写冲突引起。

结果：第二次读的值与前次读的值不同。

3. 读脏数据（dirty read）

读脏数据也称为读未提交的数据，也就是说，一事务更新的数据尚未提交，被另一事务读到，如前一事务因故要回退（ROLLBACK），则后一事务读到的数据已经是没有意义的了，即为脏数据。

原因：由于后一事务读了前一个事务写了但尚未提交的数据引起，称为写—读冲突。

结果：读到有可能要回退的更新数据。但如果前一事务不回退，那么后一事务读到的数据仍是有意义的。

6.3.3 事务隔离级别

隔离级别	丢失更新	读脏数据	不可重复读	幻影读
未提交读（read uncommitted）	是	是	是	是
提交读（read committed）	否	否	是	是
可重复读（repeatable read）	否	否	否	是
可串行读（serializable）	否	否	否	否

6.3.4 MySQL 事务隔离级别设置

1. MySQL 隔离级别的设置

MySQL 支持上述 4 种隔离级别，定义事务的隔离级别可以使用 SET TRANSACTION 语句，其语法形式如下：

```
SET SESSION TRANSACTION ISOLATION LEVEL
SERIALIZABLE
|REPEATABLE READ
|READ COMMITTED
|READ UNCOMMITTED;
```

系统变量 @@TRANSACTION_ISOLATION 中存储了事务的隔离级别，可以使用 SELECT 语句查看当前会话的事务隔离级别。

2. READ UNCOMMITTED 隔离级别

设置 READ UNCOMMITTED（读取未提交数据）隔离级别，所有事务都可以看到其他未提交事务的执行结果。该隔离级别很少用于实际应用，因为它的性能也不比其他级别好多少。读取未提交的数据，也被称为脏读（Dirty Read）。

3. READ COMMITTED 隔离级别

READ COMMITTED（读取提交的数据）是大部分数据库系统（如 SQL Server、Oracle）的默认隔离级别（但不是 MySQL 默认的）。它满足了隔离的

简单定义：一个事务只能看见已提交事务所做的改变。这种隔离级别可以避免脏读现象，但可能出现不可重复读和幻影读，因为同一事务的其它实例在该实例处理期间可能会有新 COMMIT，所以同一查询可能返回不同结果。

4. REPEATABLE READ 隔离级别

REPEATABLE READ（可重复读）是 MySQL 的默认事务隔离级别，它确保在同一事务内相同的查询语句的执行结果一致。这种隔离级别可以避免脏读以及不可重复读的现象，但可能出现幻影读现象。

5. SERIALIZABLE 隔离级别

SERIALIZABLE 是最高的隔离级别，它通过强制事务排序，使之不可能相互冲突。简单来说，它是在每个读的数据行上加上共享锁。在这个级别中，可能会导致大量的锁等待现象。该隔离级别主要用于分布式事务。

6.4 封 锁

6.4.1 锁

1. 基本锁

（1）排它锁（X 锁）

排它锁又称为写锁。若一个事务 T_1 在数据对象 R 上获得了排他锁，则 T_1 既可对 R 进行读操作，也可进行写操作。而其他任何事务不能对 R 加任何锁，因而不能进行任何操作，直至 T_1 释放了它对 R 加的锁。所以排他锁就是独占锁。

（2）共享锁（S 锁）

共享锁又称为读锁。若一个事务 T_1 在数据对象 R 上获得了共享锁，则它可对 R 进行读操作，但不能写 R 。其他事务可以也只能同时对 R 加共享锁。

2. 基本锁的相容矩阵

相 容		请 求 锁		
		S	X	—
持 有 锁 性	S	Y	N	Y
	X	N	N	Y
	—	Y	Y	Y

6.4.2 封锁协议※※※

1. 一级封锁协议

一级封锁协议内容：事务 T 在修改数据对象之前必须对其加 X 锁，直至事务结束。

2. 二级封锁协议

二级封锁协议内容：在一级封锁协议的基础上，另外加上事务 T 在读取数据对象 R 之前必须先对其加 S 锁，读完后释放 S 锁。

3. 三级封锁协议

三级封锁协议内容是：在一级封锁协议的基础上，另外加上事务 T 在读取数据 R 之前必须先对其加 S 锁，读完后并不释放 S 锁，而直至事务 T 结束才释

放。

4. 封锁协议总结

6.4.3 封锁带来的问题

1. “饿死”问题

有可能存在一个事务序列，其中每个事务都申请对某数据项加 S 锁，且每个事务在授权加锁后一小段时间内释放封锁，此时若另一个事务 T_2 欲在该数据项上加 X 锁，则将永远轮不上封锁的机会。这种现象称为“饿死”（Starvation）。

2. “活锁”问题

系统可能使某个事务永远处于等待状态，得不到封锁的机会，这种现象称为“活锁”（Live Lock）。

3. “死锁”问题

系统中两个或两个以上的事务都处于等待状态，并且每个事务都在等待其中另一个事务解除封锁，它才能继续执行下去，结果造成任何一个事务都无法继续执行，这种现象称系统进入“死锁”（Dead Lock）状态。

（1）“死锁”的预防

预防死锁有两种方法：一次加锁法和顺序加锁法。

（2）“死锁”解除

DBMS 中有一个死锁测试程序，每隔一段时间检查并发的事务之间是否发生死锁。如果发现死锁，DBA 从依赖相同资源的事务中抽出某个事务作为牺牲品，将它撤消，并释放此事务占用的所有数据资源，分配给其他事务，使其他事务得以继续运行下去，这样就有可能消除死锁。

6.5 两段封锁协议

两段封锁协议规定所有的事务应遵守下面两条规则。

（1）在对任何一个数据进行读写操作之前，事务必须获得对数据的封锁。

（2）在释放一个封锁之后，事务不再获得任何其他封锁。

6.6 MySQL 的并发控制

6.6.1 表级锁

表级锁定是指整个表被客户锁定。表级锁定包括读锁定和写锁定两种。

任何针对表的查询操作或者更新操作，MySQL 都会隐式地施加表级锁。隐式锁的生命周期（指在同一个 MySQL 会话中，对数据加锁到解锁之间的时间间隔）非常短暂，且不受数据库开发人员的控制。

MySQL 施加表级锁的命令语法形式如下：

```
lock tables 表名 read  
[表名 write]...;
```

【说明】

① read 施加表级读锁，write 施加表级写锁。

② 对表施加读锁后，客户机 A 对该表的后续更新操作将出错；客户机 B 对该表的后续查询操作以继续进行，对该表的后续更新操作将被阻塞。

③ 对表施加写锁后，客户机 A 的后续查询操作以及后续更新操作都可以

继续进行；客户机 B 对该表的后续查询操作以及后续更新操作都将被阻塞。

MySQL 解锁的命令语法格式如下：

```
unlock tables;
```

6.6.2 行级锁

行级锁比表级锁对锁定过程提供了更精细的控制。在这种情况下，只有线程使用的行是被锁定的。表中的其他行对于其他线程都是可用的。

行级锁包括共享锁（S）、排他锁（X），其中共享锁也叫读锁，排他锁也叫写锁。

共享锁。如果事务 T1 获得了数据行 R 上的共享锁，则 T1 对数据行可以读但不可以写。事务 T1 对数据行 R 加上共享锁，则其他事务对数据行 R 的排他锁请求不会成功，而对数据行 R 的共享锁请求可以成功。

排它锁。如果事务 T1 获得了数据行 R 上的排他锁，则 T1 对数据行既可读又可写。事务 T1 对数据行 R 加上排他锁，则其他事务对数据行 R 的任何封锁请求都不会成功，直到事务 T1 释放数据行 R 上的排他锁。

① 在查询语句中，为符合查询条件的记录施加共享锁，语法形式如下：

```
SELECT * FROM 表名 WHERE 条件 LOCK IN SHARE  
MODE;
```

② 在查询语句中，为符合查询条件的记录施加排他锁，语法形式如下：

```
SELECT * FROM 表名 WHERE 条件 FOR UPDATE;
```

③ 在更新（INSERT、UPDATE、DELETE）语句中，MySQL 将会对符合条件的记录自动施加隐式排他锁。

6.6.3 表的意向锁

意向锁（I）是隐式的表级锁，数据库开发人员向表的某些记录加行级锁时，MySQL 首先会自动地向该表施加意向锁，然后再施加行级锁，意向锁无需数据库开发人员维护。MySQL 提供了两种意向锁：意向共享锁（IS）和意向排他锁（IX）。

意向共享锁（IS）。事务向表的某些记录施加行级共享锁时，MySQL 会自动地向该表施加意向共享锁（IS）。也就是说，执行“SELECT * FROM 表名 WHERE 条件 LOCK IN SHARE MODE;”后，MySQL 在为表中符合条件的记录施加共享锁之前，会自动地为该表施加意向共享锁（IS）。

意向排他锁（IX）。事务向表的某些记录施加行级排他锁时，MySQL 会自动地向该表施加意向排他锁（IX）。也就是说，执行“SELECT * FROM 表名 WHERE 条件 FOR UPDATE;”后，MySQL 在为表中符合条件的记录施加排他锁之前，会自动地为该表施加意向排他锁（IX）。

【课程小结】

本次课要求掌握事务四个特性，能够判断出事务并发控制过程中引起的问题类别。本次课要求能够根据判断并发控制引发的问题类别，采用封锁协议解决并发控制问，及二级封锁协议，MySQL 下各并发锁的应用。

《数据库原理与应用》 课程教案(第 15 次课第 7 章)

编制日期： 2021 年 3 月 3 日

授课方式 (请打√)	理论课 ☒ 讨论课 ☐ 实践课 ☐ 习题课 ☐ 其他 ☐	课时 安排	2
教学单元（教学章、节或主题）： 第7章 故障恢复			
目的、要求（分了解、熟悉、掌握三个层次）： 1、了解数据库恢复的概述 2、掌握故障的种类 3、熟悉恢复的实现技术及恢复的策略 4、掌握 MySQL 下的数据备份与恢复			
重点： 1、故障种类、恢复实现技术、恢复策略 2、MySQL 数据的备份与恢复、表数据的导出与导入			
难点： 1、恢复实现技术 2、灵活应用 MySQL 下的备份与恢复语句			
教学步骤： 提问引起思考引入课程 精讲课堂内容 课堂小结 布置作业			
教具及教学手段：多媒体讲解、举例讲解			

7.2.2 系统故障

系统故障可能导致事务的两种情况：

- (1) 尚未完成的事务。
- (2) 已提交的事务。重新启动时，具体处理分为：
 - (1) 对未完成事务作 UNDO 处理；
 - (2) 对已提交事务但更新还留在内存缓冲区的事务进行 REDO 处理。

7.2.3 介质故障

介质故障恢复的方法是：

- 重新装入转储的后备副本到新的磁盘，使数据库恢复到转储时的一致状态。
- 在日志中找出转储以后所有已提交的事务。对这些已提交的事务进行 REDO 处理，将数据库恢复到故障前某一时刻的一致状态。

7.3 恢复的实现技术

7.3.1 数据备份

备份是为了支持磁盘本身发生故障时的数据库恢复。在发生介质故障时，存储在磁盘上的数据库本身甚至日志遭到破坏，将如何恢复呢？

其基本方法是定期（比如一天一次）地将数据库转储到另外分离（甚至远离）的安全存储器（磁带、光盘或远程结点等）上，这种转储过程就称为备份。

- (1) 海量转储。每次复制整个数据库。
- (2) 增量转储。

7.3.2 登记日志文件

在系统运行时，数据库与事务都在不断的变化，为了在故障后能恢复系统的正常状态，必须在系统正常运行期间随时记录下它们的变化情况，以便提供恢复所需信息。这种历史记录称为“日志”。

为保证数据库是可恢复的，登记日志文件时必须遵守 2 条原则：

- (1) 事务登记的次序必须严格按并发事务执行的时间次序。
- (2) 必须先写日志文件，后写数据库。

7.4 恢复策略

7.4.1 事务故障的恢复

- (1) 从后向前的扫描日志，找到故障事务。
- (2) 撤消该事务已做的所有更新操作。
- (3) 从正在运行的事务列表中删除该事务，释放该事务所占资源。

7.4.2 系统故障的恢复

- (1) 重新启动 OS 和 DBMS。
- (2) 从前向后扫描日志，找到故障前已提交的事务，将其事务唯一标识号记入重做（REDO）队列。同时，找出故障时未提交的事务，将其事务唯一标识号记入撤销（UNDO）队列。
- (3) 对撤销队列中的各个事务进行撤销处理，具体方法是，反向扫描日志，对每个要撤销的事务进行回退操作。
- (4) 对重做队列中的各个事务进行重做，具体方法是，正向扫描日志，对每个事务重新执行日志文件登记的操作。

7.4.3 介质故障的恢复

- (1) 修复或更换磁盘系统，并重新启动系统。
- (2) 装入最近的数据库后备副本，使数据库恢复到最近一次转储时的一致

性数据库状态。

(3) 装入有关的日志副本, 重做 (REDO) 已提交的事务。具体方法为, 扫描日志, 找出故障时已提交事务的唯一标识号, 记入重做队列; 正向扫描日志, 对重做队列中的事务重新执行日志文件中登记的操作。

7.6 MySQL 数据备份与恢复***

7.6.1 数据的备份与恢复

1. 使用 mysqldump 命令备份数据

1) 备份单个数据库或表

使用 mysqldump 备份数据库或表, 命令形式如下:

mysqldump -u 用户名 -h 主机名 -p 密码 数据库名 [表名 [表名...]] > 备份文件名.sql

2) 备份多个数据库

使用 mysqldump 备份多个数据库, 命令形式如下:

mysqldump -u 用户名 -h 主机名 -p 密码 --databases 数据库名 数据库名 ... > 备份文件名.sql

3) 备份所有数据库

使用 mysqldump 备份所有数据库, 命令形式如下:

mysqldump -u 用户名 -h 主机名 -p 密码 --all-databases > 备份文件名.sql

2. 使用 mysql 命令恢复数据

对于使用 mysqldump 命令备份后形成的.sql 文件, 可以使用 mysql 命令导入到数据库中。

使用 mysql 恢复数据, 命令形式如下:

mysql -u 用户名 -p 数据库名 < 备份文件名.sql

7.6.2 表数据的导出与导入

1. 使用 SELECT ...INTO OUTFILE 语句导出表数据

使用 MySQL 语句“SELECT ...INTO OUTFILE”可以将一个数据库中的满足条件的记录导出到指定格式的文本文件中, 该文本文件使用特殊符号分隔各个字段值。该语句的语法形式如下:

```
SELECT 语句 INTO OUTFILE '文本文件'
[FIELDS [TERMINATED BY '字符']
      [[OPTIONALLY] ENCLOSED BY '字符']
      [ESCAPED BY '字符']
]
[LINES [STARTING BY '字符串']
      [TERMINATED BY '字符串']
]
```

【说明】

① TERMINATED BY '字符': 字段分隔符, 默认是制表符'\t'。

② [OPTIONALLY] ENCLOSED BY '字符': 向字段值两边加上字段包围符。如果使用 OPTIONALLY 选项, 只在 CHAR、VARCAHR 和 TEXT 字符串类型的字段值两边添加字段包围符。

③ ESCAPED BY '字符': 设置转义字符, 默认值为'\'。

④ STARTING BY '字符串': 设置每行开头的字符, 默认情况下无任何字符。

⑤ TERMINATED BY '字符串': 设置每行的结束符, 默认值是'\n'。

2. 使用 mysqldump 命令导出表数据

用 `mysqldump` 命令不仅可以备份数据库,还可以将表数据导出到文本文件。
`mysqldump` 命令导出表数据的语法形式如下:

```
mysqldump -u root -p -T "目标路径" 数据库名 表名
```

```
[--fields-terminated-by=字符]
```

```
[--fields-enclose-by=字符]
```

```
[--fields-optionally-enclosed-by=字符]
```

```
[--fields-escaped-by=字符]
```

```
[--lines-terminated-by=字符串]
```

【说明】

① 只有指定-T 参数,才能导出纯文本文件。

② 导出生成的文件有两个,一个是包含创建表的 `CREATE TABLE` 语句的表名.sql 文件,一个是包含其数据的表名.txt 文件。

③ 目标路径必须是 MySQL 的 `secure_file_priv` 参数所指定的位置。

④ 各选项功能对应“`SELECT ...INTO OUTFILE`”语句中的各项功能。

3. 使用 `LOAD DATA INFILE` 语句导入表数据

`LOAD DATA INFILE` 语句能够快速的一个指定格式的文本文件中读取数据到一个数据库表中,它是“`SELECT ...INTO OUTFILE`”语句的反操作。其语法形式如下。

```
LOAD DATA INFILE '文本文件' INTO TABLE 表名
```

```
[FIELDS [TERMINATED BY '字符']
```

```
[[OPTIONALLY] ENCLOSED BY '字符']
```

```
[ESCAPED BY '字符']
```

```
]
```

```
[LINES [STARTING BY '字符串']
```

```
[TERMINATED BY '字符串']
```

```
]
```

```
[IGNORE n LINES]
```

4. 使用 `mysqlimport` 命令导入表数据

语法形式如下。

```
mysqlimport -u root -p 数据库名 文本文件名.txt
```

```
[--fields-terminated-by=字符]
```

```
[--fields-enclose-by=字符]
```

```
[--fields-optionally-enclosed-by=字符]
```

```
[--fields-escaped-by=字符]
```

```
[--lines-terminated-by=字符串]
```

```
[--ignore-lines=n]
```

【说明】

`--ignore-lines=n`: 忽略文本文件的前 n 行。

7.6.3 使用二进制日志文件恢复数据

日志是数据库的重要组成部分。日志文件中记录了数据库运行期间发生的变化。MySQL 日志主要分为 4 类,使用这些日志文件,可以查看 MySQL 内部发生的事情。这 4 类日志分别是:

错误日志: 记录 MySQL 服务的启动、运行或停止服务时出现的问题。

查询日志: 记录建立的客户端连接和执行的语句。

二进制日志: 记录所有更改数据的语句,可以用于数据恢复。

慢查询日志: 记录所有执行时间超过 `long_query_time` 的所有查询或不使用

索引的查询。 【课程小结】 本次课要求掌握故障的三种类型，及每种故障的恢复方法，及 MySQL 下数据备份与恢复命令实现。	
---	--